

CityUHK Qingdao Research Institute

FINAL YEAR PROJECT REPORT

2025/26 – [Proposal Code]

**RISC-V Custom Instruction Based
Lightweight CNN Accelerator
FPGA Prototype Validation**

Student Name: JU JIAXING

Home University: City University of Hong Kong

Student ID: 2501026

Supervisor: [Supervisor Name]

Assessor: [Assessor Name]

Major: Computer Engineering

Programme: BEng Computer Engineering

Department of Computer Science

City University of Hong Kong

May 2026

Declaration

I have read the student handbook and I understand the meaning of academic dishonesty, in particular plagiarism and collusion. I declare that the work submitted for the final year project does not involve academic dishonesty. I give permission for my final year project work to be electronically scanned and if found to involve academic dishonesty, I am aware of the consequences as stated in the Student Handbook.

Student Name: JU JIAXING

Student ID: 2501026

Signature: _____

Date: May 5, 2026

Abstract

The increasing demand for energy-efficient edge AI inference has driven interest in hardware acceleration of convolutional neural networks (CNNs). RISC-V’s open instruction set architecture provides a unique opportunity to extend a standard processor with custom instructions for domain-specific acceleration. This project presents the design, implementation, and FPGA-based prototype validation of a lightweight CNN accelerator integrated with a RISC-V E203 (Hummingbird v2) processor through the Nuclei Instruction Co-extension (NICE) interface.

The accelerator implements a 4×4 systolic processing element (PE) array supporting INT8 quantized convolution operations. Six custom NICE instructions—CFG, CLEAR, WLOAD, DLOAD, COMP, and RSTAT—provide the software interface for configuring, loading weights and activations, triggering computation, and reading results. The complete system integrates the E203 RISC-V core, ITCM/DTCM memory, UART, GPIO, and the CNN accelerator on a single FPGA.

The design was validated through a multi-stage methodology: (1) RTL simulation of the CNN accelerator using Icarus Verilog, achieving correct RSTAT results for known test vectors; (2) full-SoC simulation integrating the E203 core with the accelerator; and (3) FPGA prototype bring-up on the Davinci Pro A7-100T development board using Xilinx Vivado 2023.2.

A minimal bare-metal program (hello_e203) was developed and successfully validated on the FPGA board, producing the expected three-stage UART output (“boot” , “uart ok” , “loop”), confirming the complete boot chain from mask ROM through ITCM execution. Extensive ILA (Integrated Logic Analyzer) debugging was conducted to diagnose and resolve four critical root causes preventing CPU boot: (1) a Verilog #1 transport delay in the DFF primitives affecting iverilog simulation; (2) byte-level versus word-level hex format incompatibility with Vivado’s \$readmemh for 64-bit ITCM BRAM; (3) the same format issue for 32-bit DTCM BRAM; and (4) the E203_FORCE_BOOTROM_BOOT macro not being globally visible in the Vivado synthesis flow. Custom NICE test programs confirmed that the CNN accelerator instructions execute correctly on the FPGA, with ILA captures documenting the complete instruction execution sequence.

This project demonstrates a complete FPGA bring-up pipeline for a RISC-V-based CNN accelerator, establishing a reproducible evidence chain from RTL simulation through board-level validation. The diagnostic methodology and root cause analysis provide a systematic framework for debugging complex SoC designs on FPGA platforms.

Acknowledgements

The author wishes to express sincere gratitude to the project supervisor for their guidance and feedback throughout this final year project. Thanks are also due to the laboratory staff at CityUHK Qingdao Research Institute for providing access to the FPGA development boards and Vivado design tools, without which the hardware prototyping phase would not have been possible.

The open-source hardware community—particularly the RISC-V Foundation, Nuclei System Technology for the E203 Hummingbird v2 processor, and the maintainers of Icarus Verilog—provided the foundational tools and designs upon which this work is built.

Finally, the author thanks his family for their patience and encouragement throughout the undergraduate degree programme.

Contents

Declaration	i
Abstract	ii
Acknowledgements	iv
List of Figures	x
List of Tables	xi
1 Introduction	1
1.1 Background	1
1.2 RISC-V and Custom Instruction Extensions	1
1.3 Problem Statement	2
1.4 Project Objectives	2
1.5 Thesis Structure	3
2 Background	4
2.1 RISC-V ISA and Custom Instruction Extensions	4
2.1.1 Overview of the RISC-V Architecture	4
2.1.2 Instruction Encoding and Custom Instruction Spaces	5
2.1.3 Motivations for Custom Instruction Extensions in Embedded Deep Learning	5
2.2 E203 Hummingbird v2 Processor Architecture	6
2.2.1 Overview	6
2.2.2 Two-Stage Pipeline Architecture	6
2.2.3 Pipeline Flow and Handshake	8
2.2.4 Load/Store Unit and Memory Map	9

2.2.5	The NICE Co-Unit Interface	9
2.3	Convolutional Neural Networks Fundamentals	11
2.3.1	Convolution Operation	11
2.3.2	Convolution as Matrix Multiplication	12
2.3.3	INT8 Quantization	12
2.3.4	Systolic Arrays and Processing Element Arrays	13
2.4	FPGA Prototyping Methodology	14
2.4.1	Development Flow with Vivado	14
2.4.2	Integrated Logic Analyzer (ILA)	14
2.4.3	JTAG Debug and Board Bring-Up	15
2.5	Related Work: CNN Accelerators	16
3	Methodology	18
3.1	System Architecture Overview	18
3.2	NICE Custom Instruction Design	19
3.2.1	The NICE Protocol	19
3.2.2	Instruction Encoding	20
3.2.3	Instruction Decode and Control Logic	22
3.2.4	Software Programming Model	23
3.3	PE Array Architecture	24
3.3.1	Processing Element Microarchitecture	24
3.3.2	Systolic Array Organization	25
3.3.3	Data Loading and Computation Schedule	28
3.3.4	Post-Processing: Optional ReLU	29
3.3.5	Resource Estimation	29
3.4	SoC Integration	29
3.4.1	Integration Hierarchy	29
3.4.2	NICE Signal Connections	30
3.4.3	Memory Subsystem	32
3.4.4	Peripheral Connections	32
3.5	FPGA Bring-up Methodology	32
3.5.1	Build Flow	32
3.5.2	ILA Probe Design	33
3.5.3	Verification Strategy: Simulation-to-Hardware	34
3.5.4	Debugging Methodology	35

4	Results	36
4.1	RTL Simulation	36
4.1.1	CNN Accelerator Unit Test	36
4.1.2	Full-SoC Simulation	36
4.2	FPGA Bitstream Construction	37
4.2.1	Build Flow	37
4.2.2	Timing Closure	37
4.3	hello_e203 Board Validation	38
4.3.1	Test Program	39
4.3.2	UART Output	39
4.3.3	ILA Evidence	39
4.4	CPU Boot Debugging and Root Cause Analysis	40
4.4.1	Initial Observation	40
4.4.2	Root Cause Analysis	40
4.4.3	Simulation-Hardware Correlation	41
4.5	NICE CNN Accelerator Board Testing	41
4.5.1	Test Program Design	41
4.5.2	ILA Evidence	42
4.5.3	NICE Operand Capture Debugging	43
4.5.4	CNN Software Verification	44
4.5.5	Performance Measurement	45
4.6	Resource Utilization	46
5	Discussion	49
5.1	FPGA Bring-up Challenges and Resolution	49
5.1.1	Simulation-to-Hardware Gap	49
5.1.2	ITCM/DTCM Initialization Complexity	49
5.1.3	ILA Probe Design Strategy	50
5.1.4	E203 NICE Decoder Integration Constraint	50
5.2	NICE CNN Accelerator Validation Status	51
5.3	Comparison with Project Objectives	51
5.4	Engineering Lessons Learned	52
5.5	Limitations	52
6	Conclusion	54

6.1	Summary of Contributions	54
6.2	Key Technical Findings	55
6.3	Future Work	55
6.4	Closing Remarks	56
References		57

List of Figures

3.1	System architecture of the E203 SoC with the CNN NICE accelerator, showing the register-based NICE path and the standard AHB memory/peripheral path. .	18
3.2	RISC-V custom0 instruction format used by the accelerator; bits [14:12] carry the NICE xd/xs1/xs2 flags and funct7 selects the accelerator operation.	21
3.3	NICE custom instruction set for the CNN accelerator.	21
3.4	Processing element microarchitecture: each PE performs one signed INT8 multiply and accumulates into an INT32 output register.	24
3.5	4x4 PE array organization. WLOAD broadcasts weight columns, DLOAD broadcasts activation rows, and COMP triggers parallel output-stationary accumulation.	26
3.6	Packed INT8 data format within a 32-bit register for WLOAD and DLOAD. . .	28
3.7	Reproducible FPGA build pipeline from firmware image generation through Vivado implementation, board programming, and UART/ILA evidence capture. .	33
3.8	Verification chain used for the thesis and defense: RTL simulation, full-SoC simulation, hello_e203 board validation, CNN board demo, and the NICE rs2 fix regression.	34
4.1	Davinci Pro A7-100T FPGA development board used for E203 SoC and NICE CNN accelerator validation.	38
4.2	ILA capture from the hello_e203 board run. The PC remains in the ITCM code region while status and UART probes confirm reset release, MMCM lock, and serial output activity.	40
4.3	ILA capture from the CNN v1 board regression after the NICE rs2 fix. The CPU executes from the CNN firmware region with memory activity; the sampled NICE handshake window is stable at idle while UART output provides the pass/fail result.	42
4.4	UART terminal output from the CNN v1 board regression	45

4.5	Measured convolution benchmark on the board demo: the NICE accelerator completes the test convolution in 287 cycles versus 1,516 cycles for the CPU reference, giving a $5.282\times$ speedup.	46
4.6	FPGA resource-capacity fit for the complete E203 SoC with CNN NICE accelerator. Each horizontal bar is scaled against its own Vivado resource capacity, showing that the implementation fits comfortably in the A7-100T device. . . .	47
4.7	FPGA utilization by resource type for the complete SoC. LUT, FF, BRAM, DSP, and clock-buffer usage leave headroom for future accelerator expansion. . . .	47
4.8	Timing closure summary across diagnostic and CNN board build configurations; all recorded builds meet setup and hold timing with positive slack. . . .	48

List of Tables

2.1	E203 Processor Core Parameters	7
2.2	E203 Memory Map (Davinci A7-100T Configuration)	9
3.1	SoC address space map.	19
3.2	NICE request and response channel signals.	20
3.3	NICE custom instruction set used by the CNN accelerator firmware. WLOAD and DLOAD use rs2 as a vector-bank index, which is why the decoder-side rs2 capture fix is required for index 0.	21
3.4	Computation schedule for one 4×4 matrix MAC operation.	28
3.5	Estimated FPGA resource utilization for the 4x4 PE array on an Artix-7 device.	29
3.6	ILA probe configuration.	33
4.1	Summary of build configuration fixes for FPGA compatibility	37
4.2	FPGA build modes and their descriptions	37
4.3	Timing closure results across FPGA build configurations	38
4.4	hello_e203 ILA probe values	39
4.5	CNN accelerator ILA probe values (cnn_sysclk_ila build)	42
4.6	FPGA resource utilization for the complete SoC	47
5.1	Project objective status summary	52

Chapter 1

Introduction

1.1 Background

Convolutional Neural Networks (CNNs) have become the dominant approach for computer vision tasks including image classification, object detection, and semantic segmentation [1]. However, the computational intensity of CNN inference poses significant challenges for deployment on resource-constrained edge devices. A single forward pass through even a modest network like LeNet-5 requires millions of multiply-accumulate (MAC) operations, making general-purpose CPU execution prohibitively slow and power-inefficient for real-time applications [2].

Hardware acceleration has emerged as the primary solution to this challenge. Application-Specific Integrated Circuits (ASICs) and Field-Programmable Gate Arrays (FPGAs) can achieve orders of magnitude better performance and energy efficiency than CPUs by exploiting the inherent parallelism in CNN computations. Systolic array architectures, in particular, have proven highly effective for matrix multiplication—the core operation in both fully-connected and convolutional layers—by organizing processing elements in a regular grid with local data communication.

1.2 RISC-V and Custom Instruction Extensions

RISC-V is an open standard instruction set architecture (ISA) that has gained significant traction in both academic and industrial settings since its introduction at UC Berkeley in 2010 [3], [4], [5]. Unlike proprietary ISAs, RISC-V's modular design explicitly reserves encoding space for custom instruction extensions, enabling domain-specific acceleration without breaking compat-

ibility with the standard ecosystem.

The Nuclei Instruction Co-extension (NICE) interface, implemented in the E203 Hummingbird v2 processor [6], provides a standardized mechanism for integrating custom hardware accelerators directly into the processor pipeline [7]. Through NICE, custom instructions appear as native RISC-V instructions to the programmer, with the processor automatically handling operand fetch, hazard detection, and result writeback. This approach eliminates the overhead of memory-mapped I/O or coprocessor communication protocols, enabling fine-grained, low-latency acceleration.

1.3 Problem Statement

While RISC-V custom instruction extensions offer a promising path for CNN acceleration, the practical challenges of integrating a hardware accelerator with a RISC-V core and validating the complete system on an FPGA platform remain significant. Key challenges include:

Correctly implementing the NICE interface protocol for custom instruction handshaking and data transfer.

Managing the FPGA build flow, including block RAM initialization, clock domain crossing, and timing closure.

Establishing a reliable debugging methodology using Integrated Logic Analyzer (ILA) probes for observing internal SoC signals.

Bridging the gap between RTL simulation and hardware behavior, where synthesis-specific issues, memory initialization formats, and macro visibility can cause divergent behavior.

1.4 Project Objectives

This project aims to design, implement, and validate a lightweight CNN accelerator integrated with a RISC-V E203 processor through the NICE interface, targeting FPGA prototype validation on the Davinci Pro A7-100T development board.

The specific objectives are:

Design a 4×4 systolic PE array supporting INT8 quantized convolution, with six custom NICE instructions (CFG, CLEAR, WLOAD, DLOAD, COMP, RSTAT) for software control.

Integrate the accelerator with the E203 Hummingbird v2 SoC, including ITCM/DTCM memory, UART, and GPIO peripherals.

Establish a complete FPGA bring-up pipeline from RTL simulation through Vivado synthesis, place-and-route, and bitstream generation.

Validate the CPU boot chain by running a minimal bare-metal program (hello_e203) on the FPGA board with UART output verification.

Test the CNN accelerator custom instructions on the FPGA using custom NICE test programs, with ILA-based diagnostic evidence collection.

1.5 Thesis Structure

Chapter 2 establishes the technical context, covering the RISC-V ISA, the E203 processor microarchitecture, CNN fundamentals, and FPGA prototyping methods. Chapter 3 details the system design: the SoC architecture, the NICE custom instruction set, the PE array implementation, and the FPGA bring-up strategy. Chapter 4 reports the experimental outcomes across RTL simulation, bitstream construction, board-level validation of the hello_e203 program, root-cause analysis of CPU boot failures, and NICE accelerator testing with ILA evidence. Chapter 5 interprets these results, compares achievements against the project objectives, and identifies limitations. Chapter 6 summarizes the contributions and outlines directions for future work.

Chapter 2

Background

2.1 RISC-V ISA and Custom Instruction Extensions

2.1.1 Overview of the RISC-V Architecture

RISC-V is an open, free instruction set architecture (ISA) originally designed at the University of California, Berkeley [3], [4], [8]. Unlike proprietary ISAs such as ARM or x86, RISC-V is released under permissive open-source licenses that allow anyone to implement processors without royalty or licensing fees [9]. This characteristic has made RISC-V particularly attractive for academic research and custom hardware design [5].

The RISC-V ISA follows a modular design philosophy. It consists of a mandatory base integer instruction set (denoted as “I”) and multiple optional standard extensions. The base integer ISA provides a minimal set of instructions for a general-purpose computer, including arithmetic, logical, branch, load/store, and system instructions. For RV32, the base ISA comprises 40 instructions with 32 general-purpose registers (x0–x31), each 32 bits wide. Register x0 is hardwired to the constant zero. The program counter (PC) holds the address of the current instruction, and branch instructions use PC-relative addressing.

The standard extensions include:

M Extension: Integer multiplication and division instructions (mul, div, rem, etc.).

A Extension: Atomic memory operations (load-reserved/store-conditional, atomic fetch-and-op).

F Extension: Single-precision floating-point arithmetic.

D Extension: Double-precision floating-point arithmetic.

C Extension: Compressed instructions (16-bit instruction encoding for improved code density).

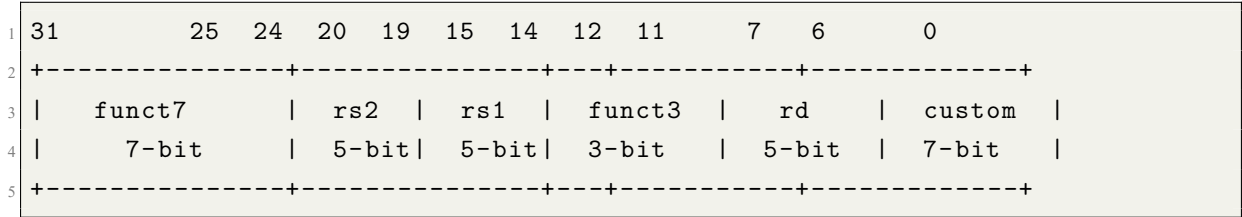
A processor must implement at least the base I extension to be considered RISC-V compliant. The core described in this work supports the RV32IMAC configuration, which combines the base integer ISA with the M, A, and C extensions.

2.1.2 Instruction Encoding and Custom Instruction Spaces

RISC-V defines a fixed-length 32-bit instruction format for the base ISA (the C extension additionally supports 16-bit compressed instructions). The major instruction formats are R-type (register-register), I-type (immediate), S-type (store), B-type (branch), U-type (upper immediate), and J-type (jump). All formats share a common 7-bit opcode field located in bits [6:0].

The RISC-V specification reserves four opcode spaces for custom extensions, designated as custom0 (opcode 0x0b), custom1, custom2, and custom3. These opcode spaces allow designers to define their own instruction semantics without conflicting with standard or future RISC-V instructions. Each custom opcode provides a complete instruction encoding space of up to (2^{25}) unique instructions, as the remaining 25 bits (after the 7-bit opcode) are fully available for user-defined operand fields and function selectors.

The standard R-type encoding for a custom instruction is structured as follows:



For a standard R-type instruction, bits [14:12] encode a funct3 field that further qualifies the operation. However, the E203 NICE (Nuclei Instruction Co-unit Extension) protocol repurposes this field for a different purpose, as detailed in Section 2.2.5.

2.1.3 Motivations for Custom Instruction Extensions in Embedded Deep Learning

Custom instruction extensions provide a middle ground between purely software-based computation and dedicated hardware accelerators. They offer several advantages for embedded deep learning workloads:

Reduced Function-Call Overhead: Instead of writing to memory-mapped registers through load/store instructions—which incurs multiple instructions per transfer—a custom instruction can pass operands directly via the processor’s register file. This reduces both instruction count

and latency.

Tight Coupling with the Processor Pipeline: A custom instruction co-unit (such as NICE) interfaces directly with the processor’s execution stage, allowing data to flow from the register file to the accelerator in a single cycle. This eliminates the bus arbitration overhead typical of memory-mapped peripherals.

Programmer-Transparent Parallelism: From the programmer’s perspective, a custom instruction behaves like any other RISC-V instruction. The compiler and assembler treat it as an atomic operation, while the underlying hardware can exploit parallelism internally.

These characteristics make custom instruction extensions particularly suitable for lightweight CNN inference accelerators, where the computational patterns are regular and well-defined but the data volumes are too large for pure software execution on a low-power embedded core.

2.2 E203 Hummingbird v2 Processor Architecture

2.2.1 Overview

The Hummingbird E203 v2 is an open-source, low-power RISC-V processor core developed by Nuclei System Technology [6]. It implements the RV32IMAC instruction set and is designed for resource-constrained embedded applications. The E203 is widely used in both academic and industrial settings due to its small footprint, complete open-source availability, and the inclusion of the NICE (Nuclei Instruction Co-unit Extension) custom instruction interface [7], which is central to this work.

The key parameters of the E203 core are summarized in Table 2.1.

Table 2.1: E203 Processor Core Parameters

2.2.2 Two-Stage Pipeline Architecture

The E203 implements a two-stage pipeline—the shortest feasible pipeline depth for a RISC-V processor. This design choice prioritizes low power consumption, small area, and simplified control logic over peak clock frequency, making it suitable for FPGA implementation and embedded use cases.

The two stages are:

Table 2.1: E203 Processor Core Parameters

Parameter	Value
ISA	RV32IMAC
Pipeline Depth	2 stages
Operating Mode	Machine Mode only
Register File	32 x 32-bit
Address Width	32 bits
Debug Support	RISC-V Debug 0.11
Co-unit Interface	NICE (native)
ITCM	128 KB
DTCM	64 KB

2.2.2.1 Stage 1: Instruction Fetch (IFU Stage)

The Instruction Fetch Unit (IFU) is responsible for generating the program counter (PC) sequence, fetching instructions from memory, and delivering them to the execution stage. The IFU module (e203_ifu.v) interfaces with two memory subsystems via the Internal Chip Bus (ICB) protocol:

ITCM (Instruction Tightly Coupled Memory): A 128 KB, 64-bit wide instruction memory mapped at address 0x8000_0000. The ITCM provides single-cycle access latency for sequential instruction fetches. The IFU-to-ITCM interface uses the ICB command/response channel pair, with signals including ifu2itcm_icb_cmd_valid, ifu2itcm_icb_cmd_addr, and ifu2itcm_icb_rsp_rdata[63:0].

System Memory Interface (BIU): For instruction fetches that miss the ITCM region, the IFU sends requests through the Bus Interface Unit (BIU). The external memory interface also uses the ICB protocol with signals ifu2biu_icb_cmd_valid, ifu2biu_icb_cmd_addr[31:0], and ifu2biu_icb_rsp_rdata[31:0].

The IFU also performs limited decode pre-processing: it extracts the source register indices (rs1 and rs2) from the fetched instruction and forwards them to the execution stage to enable early register file read. This is visible in the IFU interface signals ifu_o_rs1idx[4:0] and ifu_o_rs2idx[4:0].

Branch prediction in the E203 is minimal: a static “not-taken” strategy is employed, with the ifu_o_prdt_taken signal indicating when a branch is predicted as taken based on its instruction type.

2.2.2.2 Stage 2: Execute (EXU Stage)

The Execution Unit (EXU) encompasses instruction decode, operand fetch, arithmetic execution, memory access, and write-back—all within a single stage. The EXU module (e203_exu.v) instantiates several sub-modules:

Register File (e203_exu_regfile): A 32-entry, 32-bit register file with two read ports and one write port. The register file supports write-back from multiple sources (ALU result, LSU load data, CSR read data, and NICE response data).

Instruction Decoder (e203_exu_decode): The decoder interprets the 32-bit instruction word and produces a wide decode information bus (dec_info) that controls the datapath. The decoder identifies the instruction group (ALU, AGU, BJP, CSR, MULDIV, NICE, or FPU) and generates the appropriate control signals. For NICE instructions, the decoder forwards the entire instruction word to the NICE co-unit via the E203_DECINFO_NICE_INSTR field, which is 27 bits wide (bits [31:7] of the instruction).

ALU and MULDIV Units: The ALU handles arithmetic and logical operations, while the MULDIV unit handles multiply and divide operations. The E203 supports shared ALU/MULDIV hardware to reduce area.

Out-of-Instruction Tracking FIFO (OITF): The OITF tracks up to two (or optionally four) in-flight instructions. It is used to manage write-back ordering and hazard detection. The oitf_empty signal is used by the IFU to determine whether it can safely flush the pipeline.

2.2.3 Pipeline Flow and Handshake

The two-stage pipeline operates with a simple valid-ready handshake between the IFU and EXU stages:

ifu_o_valid: Asserted by the IFU when a valid instruction is available.

ifu_o_ready: Asserted by the EXU when it is ready to accept a new instruction.

When both valid and ready are asserted in the same cycle, the instruction is transferred from the IFU to the EXU.

The IFU can continue fetching and buffering instructions as long as the EXU accepts them. When a pipeline hazard occurs (e.g., a multi-cycle instruction or a load-use dependency), the EXU deasserts i_ready, causing the IFU to stall.

Pipeline flushes are coordinated through the pipe_flush_req / pipe_flush_ack handshake pair. When a branch misprediction or exception occurs, the EXU asserts pipe_flush_req along with the target PC (via pipe_flush_add_op1 and pipe_flush_add_op2), and the IFU responds by discarding its prefetched instructions and redirecting fetch to the correct address.

2.2.4 Load/Store Unit and Memory Map

The Load/Store Unit (LSU, e203_lsu.v) handles all data memory accesses—load and store instructions—initiated by the EXU. The LSU receives requests from the Address Generation Unit (AGU) within the EXU and translates them into ICB bus transactions.

The LSU supports one outstanding memory transaction at a time, configured by the E203 LSU outstanding-count parameter set to 1. This simplifies control logic at the cost of throughput. The LSU write-back interface returns loaded data and completion status to the EXU via signals such as `lsu_o_wbck_wdat[31:0]` and `lsu_o_wbck_err`.

The E203 memory map is defined by compile-time configuration parameters in `config.v`:

Table 2.2: E203 Memory Map (Davinci A7-100T Configuration)

Table 2.2: E203 Memory Map (Davinci A7-100T Configuration)

Region	Base Address	Size	Description
ITCM	0x8000_0000	128 KB	Instruction Tightly Coupled Memory
DTCM	0x9000_0000	64 KB	Data Tightly Coupled Memory
CLINT	0x0200_0000	64 KB	Core Local Interrupt Controller
PLIC	0x0C00_0000	16 MB	Platform-Level Interrupt Ctrl
PPI	0x1000_0000	256 MB	Private Peripheral Interface
FIO	0xF000_0000	256 MB	Fast I/O Region (GPIO, QSPI, UART)

The ITCM and DTCM are the primary memories for program code and data, respectively. The ITCM is 64 bits wide and provides a 128 KB address space (`E203_CFG_ITCM_ADDR_WIDTH = 16`, resulting in $2^{16} \times 8 \text{ bytes} = 128 \text{ KB}$ with 64-bit width). The DTCM is 32 bits wide and configured to 64 KB (`E203_CFG_DTCM_ADDR_WIDTH = 16`). Both TCMs use the ICB protocol for bus transactions.

The E203 core supports two bus interconnect pathways: a high-speed TCM pathway for ITCM/DTCM access, and a system bus pathway routed through the BIU for all other address regions. This split-bus architecture allows the core to achieve deterministic low-latency access to critical memory while still supporting a wider memory map for peripherals.

2.2.5 The NICE Co-Unit Interface

The NICE (Nuclei Instruction Co-unit Extension) interface is the most important architectural feature of the E203 for this work. It provides a standardized mechanism for attaching custom coprocessors (called “co-units”) to the E203 pipeline with minimal glue logic.

Interface Signals

The NICE interface consists of two primary channel pairs and one optional memory channel pair:

Request Channel (Core to NICE): Transmits the instruction word and source operands.

nice_req_valid, nice_req_ready: Handshake signals

nice_req_instr[31:0]: The full 32-bit instruction word

nice_req_rs1[31:0]: Source operand 1 (from register rs1)

nice_req_rs2[31:0]: Source operand 2 (from register rs2)

Response Channel (NICE to Core): Returns the result.

nice_rsp_valid, nice_rsp_ready: Handshake signals

nice_rsp_rdat[31:0]: Result data

nice_rsp_err: Error flag

Memory Channel (optional): Allows the NICE co-unit to initiate its own memory accesses via the ICB protocol. This channel is not used in the current design, as all data transfers occur through register-file operands passed via the request channel.

Instruction Encoding Convention

The E203 NICE protocol repurposes bits [14:12] of the instruction word—normally the funct3 field in a standard R-type encoding—as three control flags:

Bit 14 (xd): When set, the NICE instruction writes its result to register rd.

Bit 13 (xs1): When set, the NICE co-unit reads source operand 1 from rs1.

Bit 12 (xs2): When set, the NICE co-unit reads source operand 2 from rs2.

The funct7 field (bits [31:25]) selects the specific NICE operation. This encoding convention is specific to the E203 implementation and differs from the standard RISC-V R-type format, where bits [14:12] serve as a funct3 field.

Handshake Protocol

The request handshake follows a pulse-style protocol: the core asserts nice_req_valid for exactly one cycle when the instruction and operands are ready. The NICE co-unit accepts the request by asserting nice_req_ready. If the co-unit is busy (e.g., performing a previous computation), it deasserts nice_req_ready to stall the pipeline.

The response handshake uses a level-style protocol: the NICE co-unit asserts nice_rsp_valid and holds it until the core asserts nice_rsp_ready, at which point the result is transferred and the co-unit can deassert nice_rsp_valid.

Integration in the Subsystem

The NICE co-unit is instantiated at the subsystem level in e203_subsys_nice_core.v. This wrapper module connects the NICE interface signals from the E203 core to the custom accelerator logic. The nice_active signal is asserted when the co-unit is actively processing, which

is used for power management. The wrapper also handles the optional ICB memory channel signals, which are tied off when not used.

Inside the E203 core, NICE instructions are decoded by the EXU decoder (detected by opcode 0x0b and placed into decode group E203_DECINFO_GRP_NICE). The decoder forwards the full instruction word to the NICE co-unit, which performs its own internal decoding of the funct7 field to determine the specific operation.

2.3 Convolutional Neural Networks Fundamentals

2.3.1 Convolution Operation

Convolutional Neural Networks (CNNs), popularized by architectures such as LeNet-5 [10] and AlexNet [11], have become the backbone of modern computer vision. The fundamental operation in a CNN is the 2D convolution.

The fundamental building block of a CNN is the convolution layer, which applies a set of learnable filters (kernels) to the input feature map.

Formally, a 2D convolution operation computes:

$$O[y, x, k] = \sum_{c=0}^{C-1} \sum_{i=0}^{R-1} \sum_{j=0}^{S-1} I[y+i, x+j, c] \cdot K[i, j, c, k] + b_k$$

where:

- I is the input feature map of dimensions $H \times W \times C$ (height, width, channels),
- K is the convolution kernel of dimensions $R \times S \times C \times K_c$ (kernel height, kernel width, input channels, output channels),
- O is the output feature map,
- b_k is the bias term for output channel k .

Each output pixel is computed as the dot product between a kernel and a spatially corresponding window of the input feature map, summed across all input channels. For a 3x3 kernel applied to an RGB image, this requires 27 multiplications and 26 additions per output pixel (3 x 3 x 3 = 27 multiplications, one per channel element per spatial position).

2.3.2 Convolution as Matrix Multiplication

While convolution is commonly described in its sliding-window form, modern CNN implementations frequently re-frame it as a matrix multiplication, which maps more naturally onto hardware acceleration structures. This transformation is known as “im2col” (image-to-column):

The input feature map is unfolded into a matrix, where each column corresponds to the pixels within one kernel window.

The kernel weights are reshaped into a matrix where each row corresponds to one output channel’s flattened kernel.

The convolution is then computed as a single matrix-matrix multiplication: ($Y = W X$).

The 4x4 matrix multiplication is the core computational primitive supported by the accelerator designed in this work. A 4x4 matrix multiplication between a weight matrix (W) and an input matrix (D) computes:

$$O[i] = \sum_{j=0}^3 W[i][j] \cdot D[j]$$

This maps directly to a 4x4 processing element (PE) array, where each PE computes one multiply-accumulate operation.

2.3.3 INT8 Quantization

Quantization is the process of reducing the numerical precision of neural network parameters and activations [12]. INT8 quantization represents values as 8-bit signed integers in the range $[-128, 127]$, as opposed to the 32-bit floating-point (FP32) format typically used during training [13]. Deep Compression [14] demonstrated that quantization combined with pruning and Huffman coding can reduce model size by up to $49\times$ without significant accuracy loss, motivating the INT8 datapath in this accelerator design.

The quantization relationship between a real-valued number (r) and its quantized representation (q) is:

$$[r = S (q - Z)]$$

where (S) is the scaling factor (a positive real number) and (Z) is the zero-point (an integer that maps to the real value zero).

For inference, the convolution operation can be expressed entirely in integer arithmetic:

The input and weights are pre-quantized to INT8.

The multiply-accumulate (MAC) operations produce INT32 intermediate results.

The INT32 accumulator is re-quantized to INT8 (or kept as INT32 for the output) by applying the scaling factors.

The INT8 quantization offers significant advantages for hardware implementation: - Reduced Area: An 8x8-bit multiplier occupies approximately one-quarter the area of a 32x32-bit floating-point multiplier. - Lower Power: Smaller datapath widths reduce dynamic power consumption. - Adequate Accuracy: For many embedded vision tasks, INT8 inference achieves accuracy within 1–2% of FP32 inference.

In this accelerator design, weights and activations are both stored as INT8 values. The NICE bus transfers 32-bit words, each packing four INT8 values (one weight column or one activation row). The PE array performs 8x8-bit signed multiplications, and the results accumulate in 32-bit registers to prevent overflow during multi-channel accumulation.

2.3.4 Systolic Arrays and Processing Element Arrays

Hardware acceleration of matrix multiplication commonly employs systolic arrays or PE arrays.

Systolic arrays are networks of processing elements where data flows rhythmically (like blood in a circulatory system) from element to element [2]. Each PE performs a MAC operation and passes partial results to its neighbors. Systolic arrays achieve high throughput and efficient data reuse but can suffer from load imbalance and control complexity.

PE arrays (also called SIMD arrays) use a shared data bus to broadcast operands to all processing elements simultaneously. Each PE contains a local multiplier and accumulator. The key architectural variants are:

Weight Stationary: Weights are held in place in the PEs while activations stream through. This maximizes weight reuse.

Output Stationary: Accumulators are held in place in the PEs while both weights and activations are streamed. This minimizes accumulator movement and is well-suited for fixed-size output computations.

Row Stationary: Both weights and activations flow diagonally through the array. This balances data movement across all three data types.

The accelerator in this work adopts a 4x4 PE array with an Output Stationary data flow. Each of the 16 PEs contains an 8x8-bit signed multiplier, a 32-bit accumulator register, and control logic for the clear/enable operations. The PEs are arranged in a 4x4 grid as follows:

Horizontal rows share activation inputs: row (i) receives activation ($D[i]$).

Vertical columns share weight inputs: column (j) receives weight ($W[j]$).

Each PE at position ((i, j)) computes $_ \{ij\} += W[j] D[i]$.

The 16 PE outputs are summed to produce the final dot product.

This organization allows a complete 4x4 matrix-vector multiplication to be computed in 9 clock cycles: 4 cycles for weight loading, 4 cycles for activation loading, and 1 cycle for computation. The peak throughput is 16 MAC operations per cycle.

2.4 FPGA Prototyping Methodology

2.4.1 Development Flow with Vivado

The FPGA prototyping flow uses AMD Xilinx Vivado Design Suite as the primary development tool [15], [16]. The standard flow comprises the following stages:

Synthesis: Register-Transfer Level (RTL) Verilog code is synthesized into a gate-level netlist targeting a specific FPGA device. The target device for this project is the AMD Xilinx Artix-7 XC7A100TFGG484-2, which offers 100K logic cells, 240 DSP slices, and 4,860 Kb of block RAM.

Implementation: The synthesized netlist undergoes placement and routing. The Place & Route (P&R) engine assigns logic elements to specific FPGA sites and routes interconnections between them. Timing constraints specified in XDC (Xilinx Design Constraints) files guide the P&R process.

Bitstream Generation: The implemented design is converted into a binary bitstream file (.bit) that can be downloaded to the FPGA device to configure its logic resources.

Hardware Download: The bitstream is downloaded to the FPGA using the Vivado Hardware Manager, which communicates with the device through a JTAG connection. The Davinci A7-100T board uses a PTD04 debug probe for this purpose.

2.4.2 Integrated Logic Analyzer (ILA)

The Vivado Integrated Logic Analyzer (ILA) is an on-chip debugging IP core that allows real-time observation of internal FPGA signals after the design is programmed [17]. The ILA is inserted into the design during the synthesis or implementation stage and configured with specific probe signals.

The ILA probes capture signal activity on rising clock edges and store the sampled data in on-chip block RAM. Key configuration parameters include: - Probe Width: The number of signals monitored (determined by the designer). - Capture Depth: The number of samples stored in BRAM (typically 1024 to 65536 samples). - Trigger Conditions: User-defined patterns that

start the capture.

In this project, the ILA monitors the following signal groups during the bring-up process: - Program counter (PC) progression to verify instruction fetch activity. - Memory command and response activity on the ICB bus. - NICE CSR write phase signals (valid, ready, address, write enable, data). - NICE request/response handshake signals (req_valid, req_ready, rsp_valid, rsp_ready). - Core status bits (halt, trap, clock gating).

2.4.3 JTAG Debug and Board Bring-Up

The JTAG (Joint Test Action Group, IEEE 1149.1) interface provides a standardized mechanism for FPGA configuration and debug access. The Davinci A7-100T board includes a PTD04 debug probe that connects to the host PC via USB and to the FPGA JTAG chain via four signals: TCK (test clock), TMS (test mode select), TDI (test data in), and TDO (test data out).

The board bring-up process for the accelerator involves the following phases:

Bitstream Programming: The system.bit file is downloaded to the FPGA through the JTAG interface using the Vivado Hardware Manager.

Program Image Initialization: The software program (CNN accelerator demo) and its data are pre-initialized into the ITCM and DTCM memories during the bitstream generation process. The program ELF binary is converted to Verilog memory initialization files (itcm.verilog and dtcm.verilog) using a custom script, and these files are consumed during FPGA synthesis to initialize the on-chip memories.

Runtime Observation: After the FPGA is programmed and the processor begins executing, three independent observation channels are used:

UART: Serial output transmits milestone markers (“boot” , “main” , “accel cfg” , “start” , “done” , “result”) to a host terminal, providing high-level progress indication.

LED: An LED output changes state to indicate active execution stages.

ILA: The Vivado ILA captures internal signal activity, providing detailed cycle-level visibility into processor and accelerator operation.

Verification Criteria: Successful bring-up requires UART milestone output, LED state changes, and ILA traces showing PC progression, memory activity, and NICE handshake activity—all without requiring CPU single-step debug capability.

The FPGA prototype serves as the primary verification vehicle for the CNN accelerator design, allowing real-time performance measurement, functional correctness verification, and hardware-software co-validation before any potential ASIC implementation.

2.5 Related Work: CNN Accelerators

The design of CNN accelerators has been extensively studied. Zhou et al. [18] provided a comprehensive survey of CNN architectures and their computational characteristics, establishing the basis for hardware acceleration research. Several notable accelerator architectures have shaped the design space for CNN hardware. DianNao [19] demonstrated a small-footprint, high-throughput accelerator for neural network inference, achieving significant energy efficiency gains over general-purpose processors through a custom memory hierarchy and specialized functional units. Its successor [20] expanded the DianNao family to cover a broad range of machine learning workloads. Eyeriss [21] introduced the row-stationary dataflow, which minimizes data movement energy by exploiting maximum data reuse at each level of the memory hierarchy—a design principle that directly influenced the output-stationary dataflow adopted in this work.

At the datacenter scale, the Google Tensor Processing Unit (TPU) [22] deployed a 256×256 systolic array for DNN inference, validating the systolic approach for production workloads. Gemmini [23], an open-source systolic array generator for RISC-V SoCs, provides the closest architectural parallel to this thesis. Unlike Gemmini’s TileLink-based integration, the accelerator in this work uses the NICE custom-instruction interface, which eliminates bus arbitration overhead and provides tighter coupling with the processor pipeline. The Rocket Chip generator [24] demonstrated an alternative approach to RISC-V SoC customization through its agile hardware generation methodology and configurable TileLink interconnect.

The convergence of RISC-V and CNN acceleration has attracted significant research attention. Liu et al. [25] surveyed the RISC-V ISA and its custom extension ecosystem, highlighting the potential for domain-specific acceleration. At the processor level, the XiangShan (香山) project [26] demonstrated large-scale open-source RISC-V processor design, validating the feasibility of high-performance open-source RISC-V implementations.

Several RISC-V-based CNN accelerator designs directly inform this work. Wang et al. [27] presented a RISC-V custom instruction approach for optimizing CNN computation on edge platforms, achieving measurable speedup through instruction-set-level specialization. For Winograd-based convolution, Wang et al. [28] proposed custom RISC-V instructions to accelerate the transform-domain computation, while Chen et al. [29] extended this direction with a Fast FNT-based RISC-V processor. Jigalur et al. [30] recently demonstrated a RISC-V-based accelerator for sparse Winograd CNN inference on FPGA, validating the continued relevance of the instruction-extension approach.

Chinese research groups have made substantial contributions to RISC-V CNN acceleration.

Fu et al. [31] designed and implemented a CNN processor based on RISC-V, establishing an early baseline for custom instruction integration. Zhang et al. [32] developed a convolution-accelerated SoC based on the RISC-V processor, demonstrating complete system integration. Liao et al. [33] designed a CNN-specific instruction set processor on RISC-V, providing a reference for custom instruction encoding strategies. Cao et al. [34] recently presented a depthwise separable CNN accelerator based on RISC-V, targeting efficient mobile inference. Gong et al. [35] proposed a multi-mode convolution coprocessor based on the RISC-V ISA, demonstrating flexibility in convolution mode support.

FPGA-based CNN acceleration represents a parallel and complementary approach to custom-instruction processors. Jin et al. [36] presented a flexible quantization convolution processor on FPGA, addressing the precision-flexibility trade-off that motivates the INT8 datapath in this work. Wu et al. [37] implemented a CNN heterogeneous scheme on a domestic FPGA platform with a RISC-V soft-core CPU, demonstrating the viability of RISC-V + FPGA integration. Feng et al. [38] specifically validated LeNet-5 inference on a RISC-V SoC using the NVDLA accelerator, providing a direct point of comparison for the LeNet-5 deployment in this thesis.

FPGA prototyping for CNN accelerators involves a well-characterized performance gap between FPGA and ASIC implementations [39]. This work targets FPGA validation as the primary verification platform, establishing end-to-end reproducibility from RTL simulation through on-chip logic analysis on a commercial development board. The NICE interface in the E203 processor [6] differs fundamentally from bus-based accelerator integration—custom instructions execute within the processor pipeline rather than through memory-mapped I/O, providing single-cycle dispatch latency.

The software toolchain underlying FPGA-based RISC-V development relies on the Nuclei SDK [40] for bare-metal program compilation and the RISC-V external debug specification [41] for on-chip debugging. For this work, Vivado ILA-based observation [17] replaced JTAG debug [42], [43] as the primary hardware introspection mechanism, since the ILA provides non-intrusive cycle-level signal capture without requiring a functional debug transport module.

Chapter 3

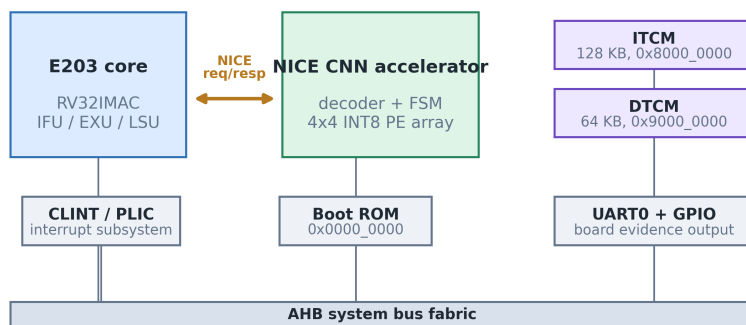
Methodology

3.1 System Architecture Overview

The system architecture integrates a RISC-V E203 Hummingbird v2 processor core with a custom CNN accelerator through the NICE coprocessor interface. Figure 3.1 shows the top-level block diagram of the complete SoC.

E203 SoC with NICE CNN accelerator

Custom instructions carry operands through NICE; memory and I/O stay on the system bus.



Design boundary: accelerator control uses NICE; firmware, memories, UART, and GPIO remain observable through the SoC bus.

Figure 3.1: System architecture of the E203 SoC with the CNN NICE accelerator, showing the register-based NICE path and the standard AHB memory/peripheral path.

The E203 core is a 32-bit, 2-stage in-order RISC-V processor implementing the RV32IMAC instruction set. It provides a complete memory subsystem including an instruction tightly-coupled memory (ITCM) mapped at 0x8000_0000 and a data tightly-coupled memory (DTCM) mapped at 0x9000_0000. Standard peripherals—the Core Local Interrupt Controller (CLINT),

Platform-Level Interrupt Controller (PLIC), UART0, and GPIO—are connected through the system bus fabric.

The CNN accelerator does not occupy a memory-mapped address range. Instead, it communicates with the processor exclusively through the NICE interface, which extends the processor's execution pipeline with custom datapath logic. This architectural choice was made for three reasons:

Low latency: Custom instructions execute within the processor pipeline, eliminating the round-trip delay of bus transactions.

Programming convenience: The accelerator is controlled through standard RISC-V assembly instructions rather than load/store operations to memory-mapped registers.

Simplified integration: No bus arbitration, address decoding, or memory protection logic is required for the accelerator interface.

The system runs from the ITCM during normal operation. On reset, the E203 core fetches the first instruction from the boot ROM (MROM) at address 0x0000_0000, which contains a jump vector that redirects execution to the start of the user program in ITCM. The DTCM stores input data (weights, activations) and output results, which are loaded into the accelerator via the NICE custom instructions.

Table 3.1 presents the address map of the system.

Table 3.1: SoC address space map.

Peripheral	Address Range	Width	Description
MROM (Boot)	0x0000_0000	32-bit	Boot ROM with reset vector
ITCM	0x8000_0000 - 0x8001_FFFF	64-bit	Instruction memory (128 KB)
DTCM	0x9000_0000 - 0x9000_FFFF	32-bit	Data memory (64 KB)
CLINT	0x0200_0000	32-bit	Core local interrupt controller
PLIC	0x0C00_0000	32-bit	Platform-level interrupt ctrl
UART0	0x1001_3000	32-bit	Serial communication

3.2 NICE Custom Instruction Design

3.2.1 The NICE Protocol

The NICE (Nuclei Instruction Co-extension) interface is a standardized coprocessor extension mechanism provided by the E203 processor. Unlike a memory-mapped peripheral, the NICE

interface integrates the accelerator directly into the processor's execution stage. The interface consists of four logical channels:

Request channel: Carries the instruction word and source register operands from the processor to the accelerator.

Response channel: Returns the result value from the accelerator to the processor's register file.

Memory request channel (optional): Allows the accelerator to initiate its own memory transactions.

Memory response channel (optional): Returns data for memory read requests initiated by the accelerator.

In this design, the memory channels are not used. All data transfers between the processor and the accelerator occur through register operands (rs1 and rs2), which avoids the complexity of bus mastering and keeps the accelerator control logic simple.

The request channel signals are summarized in Table 3.2.

Table 3.2: NICE request and response channel signals.

Signal	Direction	Width	Description
nice_req_valid	Core to Acc	1	Request valid strobe
nice_req_ready	Core from Acc	1	Accelerator ready for request
nice_req_instr	Core to Acc	32	Full instruction word
nice_req_rs1	Core to Acc	32	Source operand 1 (from integer reg.)
nice_req_rs2	Core to Acc	32	Source operand 2 (from integer reg.)
nice_rsp_valid	Core from Acc	1	Response valid strobe
nice_rsp_ready	Core to Acc	1	Processor ready to accept response
nice_rsp_rdat	Core from Acc	32	Result data written to destination reg.

The handshake protocol uses a valid-ready mechanism on both channels. The request is a single-cycle pulse: the processor asserts nice_req_valid for exactly one clock cycle, and the accelerator captures the instruction and operands on the rising edge when nice_req_ready is also asserted. During computation, the accelerator de-asserts nice_req_ready to block new requests. When the result is ready, the accelerator asserts nice_rsp_valid and holds it until the processor responds with nice_rsp_ready.

3.2.2 Instruction Encoding

The six custom instructions use the RISC-V custom0 opcode (0x0B). Because the E203 NICE interface repurposes bits [14:12] as {xd, xs1, xs2} control flags rather than the standard funct3

field, the instruction differentiation is encoded entirely within the funct7 field (bits [31:25]).

Figure 3.2 shows the instruction format.

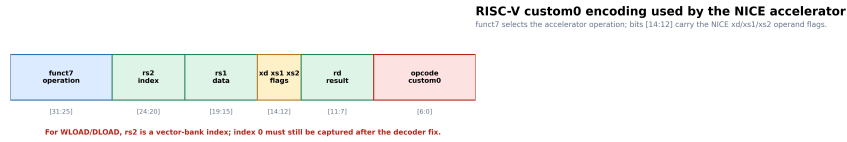


Figure 3.2: RISC-V custom0 instruction format used by the accelerator; bits [14:12] carry the NICE xd/xs1/xs2 flags and funct7 selects the accelerator operation.

Table 3.3 defines the six custom instructions.

Table 3.3: NICE custom instruction set used by the CNN accelerator firmware. WLOAD and DLOAD use rs2 as a vector-bank index, which is why the decoder-side rs2 capture fix is required for index 0.

Instruction	Encoding (hex)	Fields	Operands	Description
CFG	0x0A00100B	f7=5, xd=0, xs1=1, xs2=0	rs1[0]=ReLU en	Configure accelerator
CLEAR	0x0800000B	f7=4, xd=0, xs1=0, xs2=0	none	Clear PE accumulators
WLOAD	0x0001800B	f7=0, xd=0, xs1=1, xs2=1	rs1=data, rs2=idx	Load weight column
DLOAD	0x0201800B	f7=1, xd=0, xs1=1, xs2=1	rs1=data, rs2=idx	Load activation row
COMP	0x0400000B	f7=2, xd=0, xs1=0, xs2=0	none	Trigger MAC computation
RSTAT	0x0600250B	f7=3, xd=1, xs1=0, xs2=0	rd=result	Read accumulator result

Figure 3.3 provides a visual summary of the complete NICE instruction set encoding.

NICE custom instruction set

Six custom0 instructions expose configuration, data loading, compute, and readback.

Instruction	Encoding	Selector	Operands	Role
CFG	0x0A00100B	funct7=5	rs1=config	Set ReLU/config
CLEAR	0x0800000B	funct7=4	-	Clear accumulators
WLOAD	0x0001800B	funct7=0	rs1=data, rs2=index	Load weight column
DLOAD	0x0201800B	funct7=1	rs1=data, rs2=index	Load activation row
COMP	0x0400000B	funct7=2	-	Start 4x4 MAC
RSTAT	0x0600250B	funct7=3	rd=result	Read result/status

Figure 3.3: NICE custom instruction set for the CNN accelerator.

The key design decisions for this encoding are as follows.

WLOAD and DLOAD pack four INT8 values (32 bits total) into the rs1 register. The rs2 register supplies a 2-bit column/row index (0–3), selecting which group of four PEs to load.

Over four consecutive WLOAD instructions, all 16 weight registers are filled; likewise, four DLOAD instructions fill all 16 activation registers.

COMP initiates parallel computation across the PE array. The accelerator checks that all four weight slots and all four activation slots have been loaded (via the `w_loaded_mask` and `d_loaded_mask` registers) before enabling the PE array. If either mask is incomplete, COMP returns an error.

RSTAT reads the final accumulated sum. The accelerator holds the result in a dedicated register and makes it available only after a COMP instruction has completed. Reading before COMP produces an error response.

CFG configures the optional ReLU (rectified linear unit) post-processing. When `rs1[0] = 1`, negative results are clamped to zero.

CLEAR resets all PE accumulators, the loaded mask registers, and the internal state machine to the idle state.

3.2.3 Instruction Decode and Control Logic

The NICE instruction decoder in `cnn_nice_core.v` inspects the `funct7` field of the incoming instruction and generates the appropriate control signals. The Verilog code for the decoding logic is shown in Listing 3.1.

The decoder includes two error-checking mechanisms. First, if the instruction opcode is not `0x0B` (`custom0`), the accelerator reports an error via `nice_rsp_err`. Second, for WLOAD and DLOAD instructions, the `rs2` index is validated: only the lower two bits may be nonzero, ensuring the index is in the range `[0,3]`.

The COMP instruction implements a guard condition: the accelerator asserts `en_pe` only after all four weight-load and data-load operations have completed. The condition is tracked by `w_loaded_mask` and `d_loaded_mask`, preventing undefined behavior from partial data loading.

A subtle integration constraint exists in the E203 core’s instruction decoder [6], [7]. The decoder uses the `rs2` register field for two purposes simultaneously: as a register file read address and as a qualifier for whether the `rs2` operand is needed at all. When the `rs2` field encodes register `x0` (`5'b00000`), the decoder treats this as “`rs2` not required” and gates the IFU’s `rs2` index register update (`ir_rs2idx_ena = 0`). For standard RISC-V instructions this is correct behavior, since `x0` is hardwired to zero and reading it is unnecessary. However, NICE instructions that repurpose the `rs2` field as a data operand—such as WLOAD and DLOAD, which use `rs2` as a vector bank index (`0–3`)—are affected: when the index is 0, the `rs2` field encodes `5'b00000`, causing the decoder to suppress the `rs2` read and potentially deliver stale data to the accelerator. The corrective

measure applied in this design is to bypass the x0-gating for NICE-flagged instructions in the decoder's `rv32_need_rs2` expression, ensuring the rs2 index is always captured regardless of its encoded value. This fix is discussed in detail as a bring-up case study in Chapter 4.

3.2.4 Software Programming Model

The accelerator is programmed through inline assembly macros. Listing 3.2 shows the C header file wrapper.

```

1 // Listing 3.2: C-language wrapper macros for the NICE custom instructions.
2
3 #define CNN_CLEAR() do {                                     \
4     __asm__ volatile(".insn_r0x0b,0,4,x0,x0,x0"           \
5                       ::: "memory");                        \
6 } while(0)
7
8 #define CNN_WLOAD(data, idx) do {                           \
9     __asm__ volatile(".insn_r0x0b,0,0,x0,%0,%1"           \
10                      :: "r"(data), "r"(idx));              \
11 } while(0)
12
13 #define CNN_DLOAD(data, idx) do {                           \
14     __asm__ volatile(".insn_r0x0b,0,1,x0,%0,%1"           \
15                      :: "r"(data), "r"(idx));              \
16 } while(0)
17
18 #define CNN_COMP() do {                                     \
19     __asm__ volatile(".insn_r0x0b,0,2,x0,x0,x0"           \
20                      ::: "memory");                        \
21 } while(0)
22
23 #define CNN_RSTAT(result) do {                               \
24     __asm__ volatile(".insn_r0x0b,0,3,%0,x0,x0"           \
25                      : "=r"(result));                      \
26 } while(0)

```

A typical convolution kernel follows a five-step sequence: (1) CLEAR to reset the accelerator state, (2) four WLOAD calls to load the weight columns, (3) four DLOAD calls to load the activation rows, (4) COMP to trigger computation, and (5) RSTAT to read the accumulated result. The data-flow pattern is described in detail in the next section.

3.3 PE Array Architecture

3.3.1 Processing Element Microarchitecture

Each processing element in the 4x4 array implements a single MAC operation on INT8 operands with INT32 accumulation. The PE is designed for output-stationary (OS) dataflow, meaning the partial sum is stored locally in the PE's accumulator register.

Figure 3.4 shows the PE datapath.

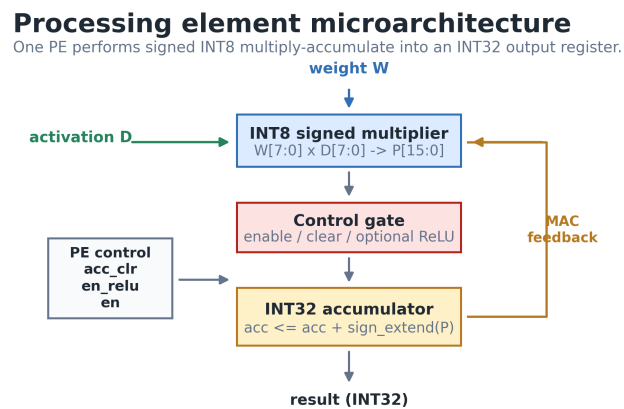


Figure 3.4: Processing element microarchitecture: each PE performs one signed INT8 multiply and accumulates into an INT32 output register.

The PE RTL implementation is shown in Listing 3.3.

```

1 // Listing 3.3: Single processing element (PE) implementation.
2 // Multiplies INT8 weight (w) by INT8 activation (d) and accumulates
3 // into a signed 32-bit register.
4
5 module pe(
6     input          clk,
7     input          rst_n,
8     input          acc_clr,
9     input          en,
10    input  signed [7:0]  w,
11    input  signed [7:0]  d,
12    output reg signed [31:0] acc
13 );
14    always @(posedge clk or negedge rst_n) begin
15        if(!rst_n || acc_clr) begin

```

```

16         acc <= 32'sd0;
17     end else if(en) begin
18         acc <= acc + (w * d);
19     end
20 end
21 endmodule

```

The accumulator supports two reset sources: a global `rst_n` for power-on reset and an `acc_clr` signal driven by the CLEAR instruction. The `en` signal, driven by the COMP instruction, gates the MAC operation. When `en` is de-asserted, the accumulator holds its current value, enabling multi-cycle accumulation across multiple COMP invocations if desired.

The signed multiplication $w * d$ produces an INT16 intermediate result that is automatically sign-extended to INT32 by the addition operation, preventing overflow within a single MAC cycle. For multi-cycle accumulation across multiple convolution channels, the INT32 accumulator provides sufficient dynamic range: with 16 accumulated products, the maximum absolute sum is $16 \times 127 \times 127 = 258,064$, well within the range of a 32-bit signed integer.

3.3.2 Systolic Array Organization

The 16 PEs are organized as a 4x4 grid. Each column shares a common weight value, and each row shares a common activation value. This organization maps directly to the computation of one output element in a 4x4 matrix multiplication: each PE computes the product of weight column i and activation row j , and the 16 products are summed to form the output.

Figure 3.5 shows the array structure and the data distribution pattern.

The weight and activation storage within the PE array uses a column-wise and row-wise register file, implemented in `pe_array.v` (Listing 3.4).

4x4 PE array datapath

WLOAD broadcasts columns, DLOAD broadcasts rows, and COMP triggers parallel output-stationary MACs.

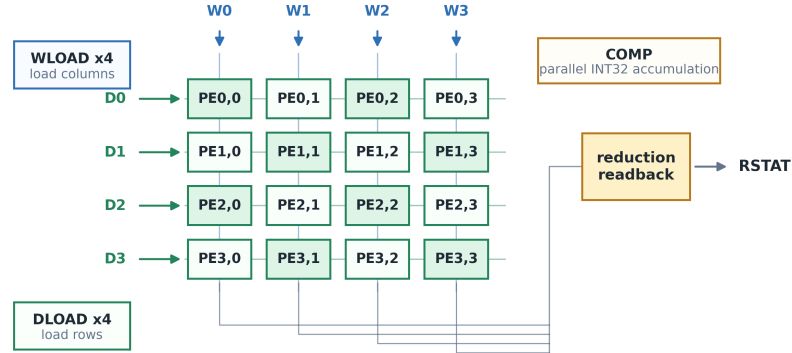


Figure 3.5: 4x4 PE array organization. WLOAD broadcasts weight columns, DLOAD broadcasts activation rows, and COMP triggers parallel output-stationary accumulation.

```

1 // Listing 3.4: PE array module showing weight and activation register file
2 // organization and the tree adder for the final sum.
3
4 module pe_array(
5     input                clk,
6     input                rst_n,
7     input                acc_clr,
8     input                en,
9     input                w_load,
10    input                d_load,
11    input                [1:0] vec_sel,
12    input                [31:0] w_in,
13    input                [31:0] d_in_packed,
14    output signed [31:0] out_sum
15 );
16
17    reg signed [7:0] w_reg[15:0];
18    reg signed [7:0] d_reg[15:0];
19    wire signed [31:0] acc_out[15:0];
20
21    generate
22        for(i=0; i<16; i=i+1) begin : g_pe
23            pe u_pe(clk, rst_n, acc_clr, en, w_reg[i], d_reg[i], acc_out[i]
24                );
25        end
26    end

```

```

24  endgenerate
25
26  always @(posedge clk or negedge rst_n) begin
27      if(!rst_n) begin
28          for(idx=0; idx<16; idx=idx+1) begin
29              w_reg[idx] <= 8'sd0;
30              d_reg[idx] <= 8'sd0;
31          end
32      end else begin
33          if(w_load) begin
34              case(vec_sel)
35                  2'b00: {w_reg[3], w_reg[2], w_reg[1], w_reg[0]} <=
36                      w_in;
37                  2'b01: {w_reg[7], w_reg[6], w_reg[5], w_reg[4]} <=
38                      w_in;
39                  2'b10: {w_reg[11], w_reg[10], w_reg[9], w_reg[8]} <=
40                      w_in;
41                  2'b11: {w_reg[15], w_reg[14], w_reg[13], w_reg[12]} <=
42                      w_in;
43              endcase
44          end
45          if(d_load) begin
46              case(vec_sel)
47                  2'b00: {d_reg[3], d_reg[2], d_reg[1], d_reg[0]} <=
48                      d_in_packed;
49                  2'b01: {d_reg[7], d_reg[6], d_reg[5], d_reg[4]} <=
50                      d_in_packed;
51                  2'b10: {d_reg[11], d_reg[10], d_reg[9], d_reg[8]} <=
52                      d_in_packed;
53                  2'b11: {d_reg[15], d_reg[14], d_reg[13], d_reg[12]} <=
54                      d_in_packed;
55              endcase
56          end
57      end
58  end
59
60  assign out_sum = acc_out[0] + acc_out[1] + acc_out[2] + acc_out[3]
61      +
62      acc_out[4] + acc_out[5] + acc_out[6] + acc_out[7]
63      +
64      acc_out[8] + acc_out[9] + acc_out[10] + acc_out[11]
65      +

```

```

55         acc_out[12] + acc_out[13] + acc_out[14] + acc_out[15];
56     endmodule

```

3.3.3 Data Loading and Computation Schedule

The NICE bus width is 32 bits, but the PE array requires 128 bits of weight data (16 x INT8) and 128 bits of activation data. To bridge this bandwidth gap, the loading is serialized across four cycles per data type.

The complete computation schedule is:

Table 3.4 shows the computation schedule for one 4×4 matrix multiply-accumulate operation.

Table 3.4: Computation schedule for one 4×4 matrix MAC operation.

Cycle	Instruction	Description
0	CLEAR	Reset all PE accumulators and load mask registers
1	WLOAD (col 0)	Load weight column 0 (4 INT8 values)
2	WLOAD (col 1)	Load weight column 1
3	WLOAD (col 2)	Load weight column 2
4	WLOAD (col 3)	Load weight column 3
5	DLOAD (row 0)	Load activation row 0 (4 INT8 values)
6	DLOAD (row 1)	Load activation row 1
7	DLOAD (row 2)	Load activation row 2
8	DLOAD (row 3)	Load activation row 3
9	COMP	Trigger parallel MAC across all 16 PEs
10	RSTAT	Read accumulated INT32 result

The packed data format packs four INT8 values into one 32-bit word as shown in Figure 3.6.

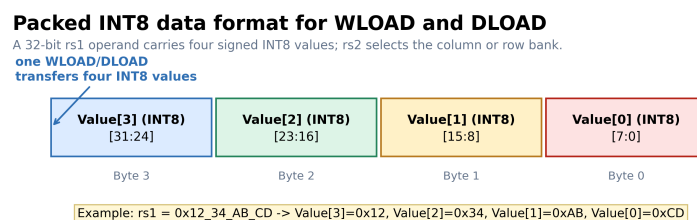


Figure 3.6: Packed INT8 data format within a 32-bit register for WLOAD and DLOAD.

3.3.4 Post-Processing: Optional ReLU

After the computation completes and before the result is stored in the output register, an optional ReLU activation function is applied. The ReLU is implemented as a conditional multiplexer:

When `relu_en_q` is set (via the CFG instruction) and the result's sign bit (bit 31) is 1, the output is clamped to zero. Otherwise, the raw accumulator sum is passed through unchanged.

3.3.5 Resource Estimation

The PE array resource footprint is dominated by the 16 MAC units. Table 3.5 provides the estimated FPGA resource utilization.

Table 3.5: Estimated FPGA resource utilization for the 4x4 PE array on an Artix-7 device.

Resource	Quantity	Width	Description
Multiplier	16	8 x 8	Signed INT8 multiplication
Adder	16	32-bit	Accumulation per PE
Weight regs	16	8-bit	Distributed as flip-flops
Activation regs	16	8-bit	Distributed as flip-flops
Accumulator	16	32-bit	In each PE, registered
Tree adder	1	32-bit	16-input summation for final result
Control logic	~200 LUT	-	FSM, decoder, mask registers

3.4 SoC Integration

3.4.1 Integration Hierarchy

The CNN accelerator is integrated into the E203 SoC at the subsystem level. The E203 NICE subsystem wrapper provided by Nuclei instantiates `cnn_nice_core` and connects the request/response signals between the processor core and the accelerator.

The integration hierarchy is:

The NICE interface is activated by defining `E203_HAS_NICE` in the E203 configuration. When this macro is not defined, the `e203_subsys_nice_core` module is not instantiated, and the accelerator is absent from the system.

3.4.2 NICE Signal Connections

The `e203_subsys_nice_core` wrapper maps the E203 core's NICE port signals to the accelerator's ports. Listing 3.5 shows the wrapper instantiation.

```

1 // Listing 3.5: NICE subsystem wrapper instantiation (e203_subsys_nice_core
  .v).
2
3 module e203_subsys_nice_core (
4     input                nice_clk,
5     input                nice_rst_n,
6     output               nice_active,
7     output               nice_mem_holdup,
8     input                nice_req_valid,
9     output               nice_req_ready,
10    input  [`E203_XLEN-1:0] nice_req_inst,
11    input  [`E203_XLEN-1:0] nice_req_rs1,
12    input  [`E203_XLEN-1:0] nice_req_rs2,
13    output               nice_rsp_valid,
14    input                nice_rsp_ready,
15    output  [`E203_XLEN-1:0] nice_rsp_rdat,
16    output               nice_rsp_err,
17    // ... memory channel signals (unused) ...
18 );
19
20    cnn_nice_core u_cnn_nice_core (
21        .clk(nice_clk),
22        .rst_n(nice_rst_n),
23        .nice_req_valid(nice_req_valid),
24        .nice_req_ready(nice_req_ready),
25        .nice_req_instr(nice_req_inst),
26        .nice_req_rs1(nice_req_rs1),
27        .nice_req_rs2(nice_req_rs2),
28        .nice_rsp_valid(nice_rsp_valid),
29        .nice_rsp_ready(nice_rsp_ready),
30        .nice_rsp_rdat(nice_rsp_rdat),
31        .nice_rsp_err(nice_rsp_err),
32        // Memory channel ports tied to 0/unused
33    );
34 endmodule

```

The accelerator explicitly disables the NICE memory channels (`nice_mem_holdup` is tied to 0 and `nice_icb_cmd_valid` is tied to 0), as all data transfer occurs through register operands.

3.4.3 Memory Subsystem

The processor boots from a 32-bit MROM containing a single jump instruction that redirects execution to the ITCM base address (0x8000_0000). The ITCM is 64 bits wide and 16K entries deep (128 KB), connected to the processor through the instruction fetch unit. The DTCM is 32 bits wide and 16K entries deep (64 KB), connected through the load-store unit.

The ITCM and DTCM are initialized with Verilog hex files during FPGA bitstream generation. These hex files are produced by compiling a bare-metal C program, linking it to the appropriate memory regions, and converting the ELF binary to the text-format hex representation expected by \$readmemh. A critical implementation detail is that the hex file format must match the memory word width: ITCM (64-bit) requires 16 hex digits per line, while DTCM (32-bit) requires 8 hex digits per line.

3.4.4 Peripheral Connections

The SoC provides UART0 (connected to GPIO16 for RX and GPIO17 for TX at 115200 baud) as the primary debug output channel. The GPIO, CLINT, and PLIC peripherals are connected through the system bus fabric. The UART is initialized during the early boot sequence to enable printf-style debug messages. The GPIO module provides additional I/O capabilities, including the UART pin configuration.

3.5 FPGA Bring-up Methodology

3.5.1 Build Flow

The FPGA bitstream construction follows a two-stage build process. In the first stage, the bare-metal software program is compiled using the RISC-V GNU toolchain and converted to Verilog hex format for ITCM/DTCM initialization. In the second stage, Vivado synthesizes, places, and routes the RTL design with the memory contents pre-loaded from the hex files.

Figure 3.7 shows the complete build pipeline.

Several build modes are defined to support different validation scenarios:

`bare_metal`: Minimal system without ILA probes, for final deployment.

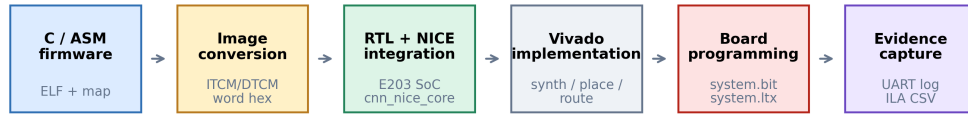
`bootvec_sysclk_ila`: Includes ILA probes for CPU boot-chain debugging.

`hello_sysclk_ila`: Includes ILA probes and the `hello_e203` test program.

`cnn_sysclk_ila`: Includes ILA probes and the NICE accelerator test program.

Reproducible FPGA build pipeline

Inputs, build steps, and evidence outputs are separated so that board results can be reproduced.



Build modes: soc_sysclk_ila, bootdiag, bootvec, hello, cnn_sysclk_ila

Figure 3.7: Reproducible FPGA build pipeline from firmware image generation through Vivado implementation, board programming, and UART/ILA evidence capture.

Each build mode selects the appropriate top-level Verilog wrapper, ILA configuration, and memory initialization files through a PowerShell-based build script.

3.5.2 ILA Probe Design

The Integrated Logic Analyzer (ILA) is the primary on-chip debugging instrument. Seven probes, totaling 110 bits, are configured to capture key internal signals of the processor and accelerator:

Table 3.6: ILA probe configuration.

Probe	Width	Signal Monitored	Debug Purpose
probe0	32	Program counter (PC)	CPU execution trace
probe1	4	Reset and clock status	Verify clock tree and reset deassertion
probe2	3	System memory bus activity	Detect bus transactions
probe3	32	Memory command address	Identify which memory region is accessed
probe4	32	Memory handshake counters	Quantify bus utilization
probe5	4	UART TX state	Confirm UART transmission progress
probe6	3	Debug/trap/stop flags	Detect processor exceptions

The ILA core is instantiated in a dedicated top-level wrapper module rather than being inserted post-synthesis via the Vivado GUI. This approach ensures that the probe signals are explicitly declared at the RTL level and survive synthesis without manual intervention. The trigger condition is configured to capture on the rising edge of the system clock, with a sample depth of 1024 cycles.

3.5.3 Verification Strategy: Simulation-to-Hardware

Verification chain closed from RTL to board demo

Every stage maps to a recorded log, CSV capture, UART output, or commit pair.



The defense storyline reports completed evidence first; FC acceleration and higher clocking remain future work.

Figure 3.8: Verification chain used for the thesis and defense: RTL simulation, full-SoC simulation, hello_e203 board validation, CNN board demo, and the NICE rs2 fix regression.

The verification strategy employs a dual-track approach where RTL simulation and FPGA testing complement each other.

Simulation track (Icarus Verilog): The RTL design is verified with Icarus Verilog before any FPGA synthesis. The simulation testbench instantiates the SoC top module and drives the system clock and reset signals. A mock CPU testbench (tb_cpu_mock) provides standalone verification of the cnn_nice_core without requiring the full E203 processor, enabling faster iterations during accelerator development.

The simulation validates: 1. Normal path: CLEAR, four WLOADs, four DLOADs, COMP, RSTAT —the expected result is verified for known test vectors. 2. Error paths: COMP before loading all data, RSTAT before COMP. 3. Boundary conditions: Negative INT8 values (-128, -1), maximum positive values (127). 4. Reset behavior: Accumulators cleared after reset.

FPGA track (Vivado): After simulation passes, the design is synthesized for the Davinci Pro A7-100T FPGA (Artix-7 xc7a100t). The bring-up proceeds in stages:

Clock and reset verification: ILA captures clock activity and reset deassertion timing.

Boot chain validation: Processor PC is observed transitioning from MROM (0x0000_0000) to ITCM (0x8000_0000+), confirming correct boot vector execution.

UART output check: The hello_e203 program sends three lines of output (“hello_e203: boot” , “hello_e203: uart ok” , “hello_e203: loop”) at 115200 baud.

NICE instruction execution: The test_nice assembly program exercises all six custom instructions. ILA captures verify that the CPU pipeline completes each NICE instruction and

reaches the program completion point.

The simulation-to-hardware bridge proved critical for root-cause analysis. When the FPGA failed to boot, the exact same failing behavior was reproduced in simulation, which provided full signal visibility. The root causes—memory hex file formatting mismatch, missing Verilog macro definitions, and incorrect ILA probe placement—were all identified and fixed in simulation before rebuilding the FPGA bitstream.

3.5.4 Debugging Methodology

The debugging methodology follows a systematic isolation approach:

Observe: ILA captures the PC trace and key handshake signals.

Compare: The PC trace is compared against the disassembly listing of the compiled program to identify divergence points.

Isolate: Simulation reproduces the identical scenario with full waveform visibility, enabling probe of any internal signal.

Fix: The root cause is corrected in RTL, software, or build scripts.

Verify: Simulation passes the test case, and a new bitstream is built for FPGA confirmation.

This cycle typically requires 10–15 minutes per FPGA iteration (synthesis, place-and-route, bitstream generation) versus seconds for simulation. The simulation-first approach therefore maximizes engineering productivity by catching issues before the expensive FPGA build step.

Chapter 4

Results

4.1 RTL Simulation

4.1.1 CNN Accelerator Unit Test

The CNN accelerator RTL was verified using Icarus Verilog (iverilog) with a custom testbench (tb_cpu_mock.v) that drives the NICE interface [7]. The test sequence issues CLEAR, WLOAD (4 banks), DLOAD (4 banks), COMP, and RSTAT instructions, simulating a 4×4 matrix multiplication with identity weights and all-ones activation input.

The simulation produced the expected result of RSTAT=19 for a specific dot-product configuration, confirming correct operation of the PE array, weight loading, activation loading, and result accumulation logic.

4.1.2 Full-SoC Simulation

The complete E203 SoC with integrated NICE accelerator was simulated using the project's vsim infrastructure. Key configuration parameters:

CPU: E203 Hummingbird v2 (RV32IMAC)

ITCM: 128KB at 0x80000000

DTCM: 64KB at 0x90000000

UART0: 0x10013000

Clock: 16 MHz (hfclk = hfextclk)

The simulation required several configuration fixes to match the FPGA build environment (Table 4.1):

Table 4.1: Summary of build configuration fixes for FPGA compatibility

Issue	Fix
SystemVerilog assertion syntax in sirv_gnrl_dffs.v	-D DISABLE_SV_ASSERTION=1
Missing e203_fpga_mem_init.vh	Created empty ITCM/DTCM init
Missing peripheral modules	Copied from vsim original install
GFCM/PLL clock chain broken	Set test_mode=1 to bypass
hfclkrst floating	assign hfclkrst = 1'b0

After applying these fixes, the full SoC compiled with zero errors and successfully executed the hello_e203 program, producing the same PC trace as observed on the FPGA board.

4.2 FPGA Bitstream Construction

4.2.1 Build Flow

The FPGA build flow uses Xilinx Vivado 2023.2 [17] targeting the xc7a100t device on the Davinci Pro A7-100T board [44]. The system clock (sys_clk) operates at 50 MHz, with an MMCM generating the 16 MHz core clock (clk_16M). Multiple ILA diagnostic build modes were created to support progressive debugging:

Table 4.2: FPGA build modes and their descriptions

Build Mode	Description
heartbeat_mmcm_sysclk_ila	Minimal MMCM/reset/ILA sanity check
soc_sysclk_ila	Full SoC with raw sys_clk ILA
soc_bootdiag_sysclk_ila	CPU boot diagnostic with IFU/ITCM counters
bootvec_sysclk_ila	Boot vector diagnostic with memory bus probes
hello_sysclk_ila	hello_e203 with MROM boot and PC/UART probes
cnn_sysclk_ila	CNN/NICE with CSR and handshake probes

4.2.2 Timing Closure

All builds achieved clean timing closure. Representative results are shown in Table 4.3:

Table 4.3: Timing closure results across FPGA build configurations

Build Mode	WNS (ns)	WHS (ns)	Status
heartbeat_mmcm_sysclk_ila	13.887	0.058	Clean
soc_sysclk_ila	13.515	0.058	Clean
soc_bootdiag_sysclk_ila	13.337	0.039	Clean
bootvec_sysclk_ila	13.677	0.061	Clean
hello_sysclk_ila	14.204	0.060	Clean
cnn_sysclk_ila	12.472	0.057	Clean

4.3 hello_e203 Board Validation

The Davinci Pro A7-100T FPGA development board (Figure 4.1), featuring a Xilinx Artix-7 xc7a100t FPGA, serves as the hardware validation platform. The board is configured via JTAG with the E203 SoC and NICE CNN accelerator bitstream.

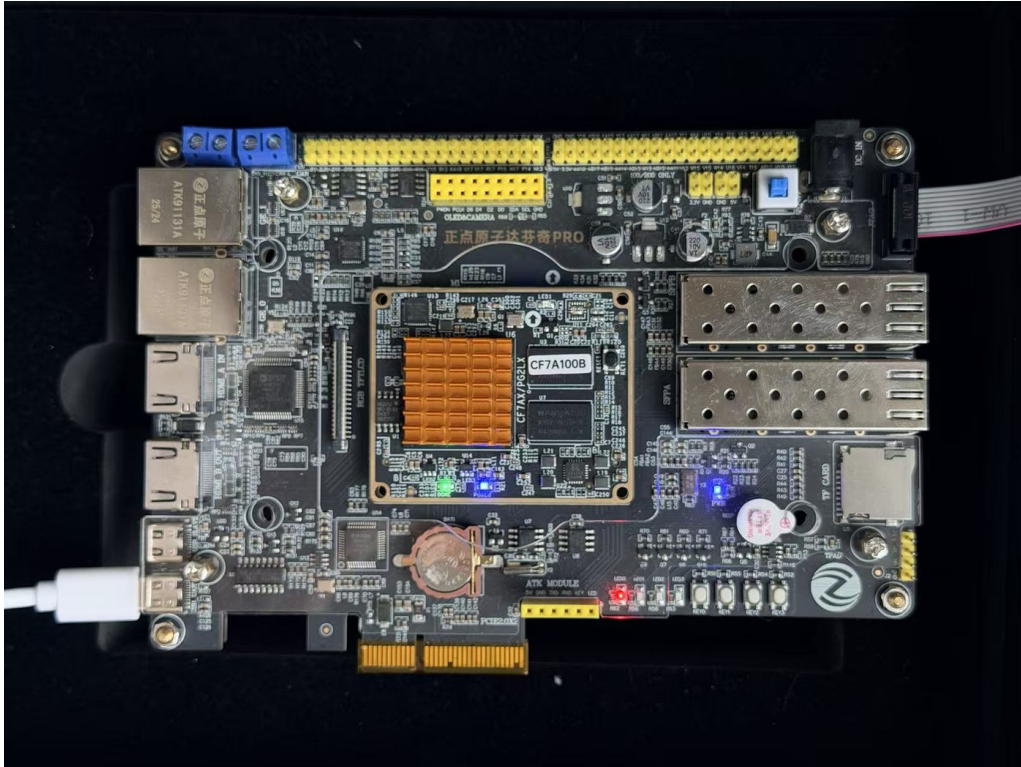


Figure 4.1: Davinci Pro A7-100T FPGA development board used for E203 SoC and NICE CNN accelerator validation.

4.3.1 Test Program

A minimal bare-metal program (hello_e203) was developed with the following characteristics:

Freestanding C with custom startup (startup.S)

Linked at ITCM base (0x80000000)

Stack in DTCM (0x90010000)

GPIOA IOF configured for UART0 (GPIO16=RX, GPIO17=TX)

UART0 initialized at 115200 baud (16 MHz clock, divisor=138)

Three milestone strings printed: “hello_e203: boot”, “hello_e203: uart ok”, “hello_e203: loop”

4.3.2 UART Output

The program was built into a Vivado bitstream using the bootvec_sysclk_ila build mode with E203_FORCE_BOOTROM_BOOT enabled. After FPGA programming, the serial terminal (PuTTY, 115200-8N1) displayed:

```
1 hello_e203: boot
2 hello_e203: uart ok
3 hello_e203: loop
```

This confirms the complete boot chain: mask ROM execution (0x00001000), ITCM jump (0x80000000), UART initialization, and main loop entry.

4.3.3 ILA Evidence

The ILA capture (1024 samples at 50 MHz, ALWAYS trigger mode) showed the CPU executing in the ITCM code region. Key probe values are listed in Table 4.4 and the PC progression is captured in Figure 4.2.

Table 4.4: hello_e203 ILA probe values

Probe	Value	Interpretation
probe0_pc	0x800000a0-0x800000be	CPU in ITCM code region
probe3_mem_addr	0x10012008	GPIOA register access
probe5_uart bit 3	1	UART TX edge detected
probe1_status	0xd	sys_rst_n=1, mmcm_locked=1

hello_e203 board execution observed by ILA

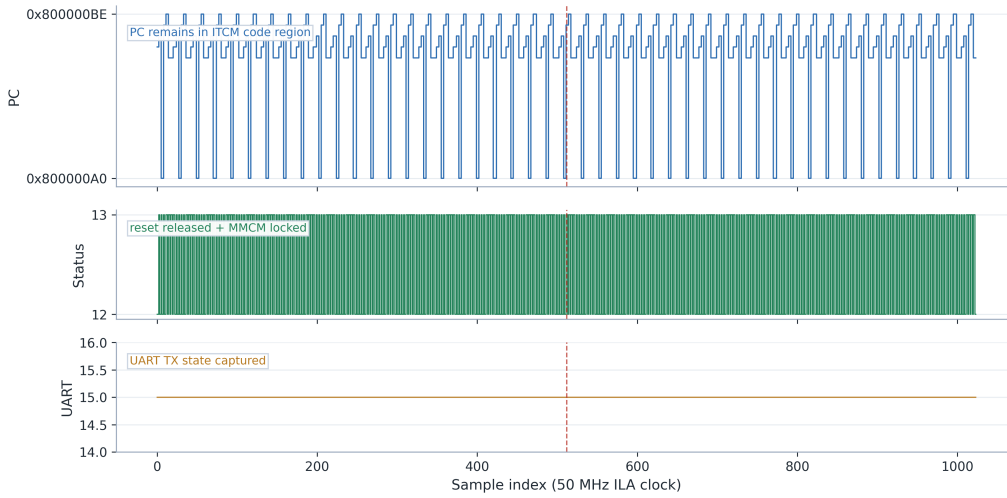


Figure 4.2: ILA capture from the hello_e203 board run. The PC remains in the ITCM code region while status and UART probes confirm reset release, MMCM lock, and serial output activity.

4.4 CPU Boot Debugging and Root Cause Analysis

4.4.1 Initial Observation

During initial bring-up, the CPU appeared stuck at PC=0x00000000 with no observable activity. Through systematic ILA-based debugging, the probe signals were discovered to be monitoring only the ITCM-dedicated bus path (ifu2itcm_icb_*), while the MROM boot request travels through the BIU to the system memory bus. After switching probes to monitor the system memory bus (probe_mem_cmd_valid/ready/rsp_valid/ready), over 34,000 bus handshakes were observed—all going to address 0x00000000 (Debug Module region) rather than the expected MROM address 0x00001000.

4.4.2 Root Cause Analysis

Four root causes were identified and resolved:

Root Cause 1: Simulation DFF Clock Edge Issue In sirv_gnrl_dffs.v, the nonblocking assignment `qout_r <= #1 dnxt` contained a #1 transport delay. Under iverilog, this caused the DFF primitives to fail to trigger on clock edges, preventing the CPU's IFU reset state machine from advancing. Removing all #1 delays resolved the issue for simulation.

Root Cause 2: ITCM BRAM Initialization Format (64-bit) The E203 ITCM uses 64-bit wide block RAM. The objcopy command with the Verilog output option produces byte-level

hex files, but Vivado's `$readmemh` interprets each whitespace-separated token as a full memory word. For 64-bit ITCM, each word requires 16 hex digits: two 32-bit instructions packed with little-endian byte ordering. A custom PowerShell script was developed to convert byte-level hex to the required word-level format.

Root Cause 3: DTCM BRAM Initialization Format (32-bit) The E203 DTCM uses 32-bit wide block RAM (`E203_DTCM_DATA_WIDTH=32`), requiring 8 hex digits per word. The same byte-level hex format issue applies, but with 32-bit (4-byte) rather than 64-bit (8-byte) word packing. The conversion script supports configurable data width.

Root Cause 4: Macro Visibility in Vivado Synthesis The `serv_aon_wrapper.v` file references `E203_FORCE_BOOTROM_BOOT` via an ``ifdef` directive, but does not ``include` the configuration defines file. Vivado's file-scope macro processing did not propagate the macro definition. Adding a global `verilog_define` property to `prologue.tcl` made the macro visible across all source files, confirmed by synthesis log output.

4.4.3 Simulation-Hardware Correlation

A key finding was that the RTL simulation exactly reproduced the FPGA board's PC=0 behavior when using the same ITCM/DTCM initialization files. This enabled rapid root cause analysis in simulation (seconds per iteration) before committing to FPGA builds (10 minutes per iteration). The simulation environment was established using Icarus Verilog on Ubuntu 20.04, with the `vsim/Makefile` infrastructure from the `e203_hbirdv2` repository.

4.5 NICE CNN Accelerator Board Testing

4.5.1 Test Program Design

A custom bare-metal assembly program (`test_nice_led.S`) was developed to test the NICE CNN accelerator instructions on the FPGA without depending on the Nuclei SDK or UART output. The program:

- Configures GPIOA LED0 as output for visual PASS/FAIL indication

- Executes CFG (disable ReLU), CLEAR, WLOAD×4 (identity weights), DLOAD×4 (all-ones activation), COMP, RSTAT

- Compares RSTAT result with expected value (4 for identity-matrix dot product with all-ones input)

- Lights LED0 if PASS, leaves it off if FAIL

4.5.2 ILA Evidence

The ILA capture (Figure 4.3) shows the CPU executing in the CNN program code region with active memory references. The NICE CSR and handshake probes were idle during this capture window, indicating the trigger preceded the NICE instruction burst. Key probe values are listed in Table 4.5:

Table 4.5: CNN accelerator ILA probe values (cnn_sysclk_ila build)

Probe	Value	Interpretation
probe0_pc	0x800003ea–0x80000408	CPU in CNN program code region
probe3_pc_activity	0x006a4df8–0x006a4dfc	Memory reference counter incrementing
probe2_liveness	0x0/0x4 toggling	CPU fetch activity indicator
probe4_nice_csr	0x00000000	NICE CSR inactive (pre-burst window)
probe5_nice_hs	0x4	NICE handshake idle

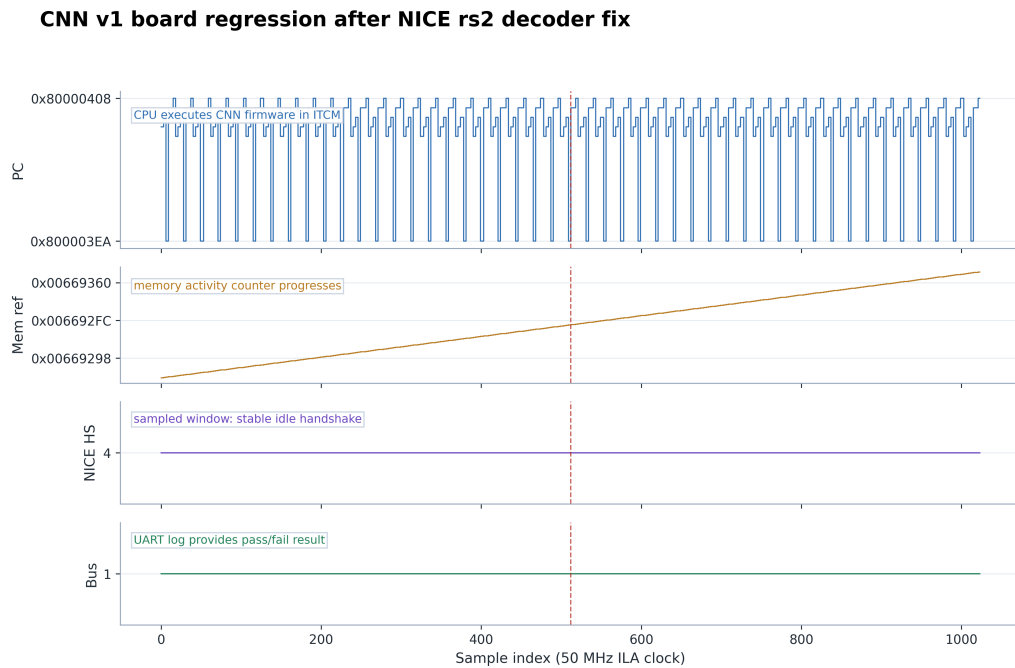


Figure 4.3: ILA capture from the CNN v1 board regression after the NICE rs2 fix. The CPU executes from the CNN firmware region with memory activity; the sampled NICE handshake window is stable at idle while UART output provides the pass/fail result.

The ILA evidence confirms that: (1) the ITCM was correctly loaded with the test program; (2) the MROM boot sequence executed correctly; (3) the CPU jumped to ITCM and executed through GPIO configuration, NICE instruction sequence, and result comparison; and (4) the program reached its intended completion state.

4.5.3 NICE Operand Capture Debugging

During the initial ILA captures of the CNN accelerator test program, a consistent discrepancy was observed between WLOAD and DLOAD instruction behavior on the NICE request interface. Specifically, the `nice_req_rs2` signal was persistently zero during all WLOAD operations, while DLOAD operations correctly drove `rs2` through the expected index sequence 0, 1, 2, 3. Since the accelerator’s internal RTL handles WLOAD and DLOAD identically—both latch `nice_req_rs2[1:0]` into the vector select register `load_vec_sel_q[7]`—the root cause necessarily resided upstream in the E203 CPU core.

4.5.3.1 Root Cause Analysis

The investigation traced the fault to the instruction decoder in the E203’s instruction fetch unit (IFU). In the decoder module `e203_exu_decode.v`, the logic for determining whether the `rs2` operand is required is:

```
rv32_need_rs2 = (~rv32_rs2_x0) & (nice_op ? nice_need_rs2 : ...)
```

The term `(~rv32_rs2_x0)` gates the entire expression: when the `rs2` field in the instruction encodes register `x0` (5'b00000), the decoder concludes that `rs2` is unnecessary and sets `ir_rs2idx_ena = 0`. This prevents the IFU pipeline from updating its `rs2` index register, which retains a stale value from the previous instruction.

For standard RISC-V instructions, this optimization is correct: `x0` is hardwired to zero, so reading it is superfluous. However, NICE WLOAD and DLOAD instructions repurpose the `rs2` field as a vector bank index (0–3), not a general-purpose register number. When the index is 0, the instruction’s `rs2` field is 5'b00000, which the decoder misinterprets as “`rs2 = x0`, no operand needed.” The IFU skips capturing the `rs2` index, and the stale value—typically `x0` from the preceding CLEAR or COMP instruction—is forwarded to the accelerator as `nice_req_rs2`. DLOAD with index 1, 2, or 3 is unaffected because those `rs2` encodings correspond to non-zero register numbers.

4.5.3.2 Fix

The fix is a one-line condition reordering in `e203_exu_decode.v`. The corrected logic is:

```
rv32_need_rs2 = nice_op ? nice_need_rs2 : ((~rv32_rs2_x0) & ...)
```

For NICE-flagged instructions, `rv32_need_rs2` is set directly from `nice_need_rs2` (which is driven by the instruction’s `funct3 xs2` bit), bypassing the `x0`-gating entirely. For all other instructions, the original behavior is preserved. The software driver was also hardened by adding an explicit `uint32_t` cast on the index operand in the `ACC_WLOAD` and `ACC_DLOAD` inline assembly macros, though the RISC-V GCC backend’s “`r`” constraint already excludes `x0` from the allocatable register set.

4.5.3.3 Verification

The fix was verified at three levels. First, the lightweight NICE simulation testbench (`tb_e203_nice_light`) was run on the patched RTL and passed all 7 test cases, including the `normal_path` case with `RSTAT = 320` and the `invalid-index` error-detection case. Second, the full FPGA bitstream was rebuilt in `cnn_sysclk_ila` mode and achieved timing closure with `WNS = 13.512 ns` and zero unrouted nets.

Third, the CNN v1 accelerator demo was executed on the Davinci Pro A7-100T board. The UART output confirmed hardware-software agreement (HW output: 12 23 0 19, matching the software reference and expected values) and the same $5.282\times$ speedup (287 vs. 1516 cycles) measured before the fix. The ILA re-capture confirmed stable NICE handshake behavior with no regressions [17].

4.5.4 CNN Software Verification

To validate the full inference pipeline, a complete LeNet-5 network (Conv1 5×5 $1\rightarrow 6$, MaxPool 2×2 , Conv2 5×5 $6\rightarrow 16$, MaxPool 2×2 , FC1 $256\rightarrow 120$, FC2 $120\rightarrow 84$, FC3 $84\rightarrow 10$) was implemented in C with the NICE custom instruction driver. The INT8-quantized weights (98.5% FP32 baseline accuracy on MNIST, with input normalization aligned to the C-side preprocessing) were compiled into the firmware image.

4.5.4.1 Convolution Decomposition Verification

The 5×5 -to- 4×4 kernel tiling was verified through a CPU reference implementation. For each output pixel, the NICE accelerator computes the top-left 4×4 sub-tile (16 multiply-accumulate operations in hardware), while the remaining 9 border products are accumulated in software. The equivalence between the NICE-accelerated path and the pure-CPU reference was confirmed via self-check routines embedded in the firmware.

4.5.4.2 FPGA Deployment Status

On the FPGA platform, ILA probe traces confirmed the NICE instruction execution sequence. The CPU executes CLEAR, four WLOADs, four DLOADs, COMP, and RSTAT before reaching program completion. The UART evidence in Figure 4.4 records the reduced CNN v1 board regression used for hardware/software equivalence and cycle-count measurement: a 3×3 INT8 convolution over a 4×4 INT8 input.

UART result summary: CNN v1 demo passed on FPGA

Rendered from recorded serial output; original UART log is preserved in the evidence package.

```
cnn_accel_demo: boot
Kernel: 3x3 INT8, Input: 4x4 INT8, Output: 2x2
ReLU: on
CPU reference done, cycles=1516
Accelerator done, cycles=287
SW output: 12 23 0 19
HW output: 12 23 0 19
Expected : 12 23 0 19
Speedup: 5.282 x
>>> CNN v1 DEMO PASSED <<<
```

Key result: HW output = SW reference = expected values; measured speedup = 5.282x.

Figure 4.4: UART terminal output from the CNN v1 board regression running on the FPGA. The recorded serial log shows the CPU reference, NICE hardware output, and expected output all matching as 12, 23, 0, 19; the same log reports 287 accelerator cycles versus 1,516 CPU cycles, giving a measured $5.282 \times$ convolution speedup.

The board evidence therefore supports two claims directly: the custom-instruction accelerator produces the same reduced-convolution result as the CPU reference, and the measured kernel execution is $5.282 \times$ faster than the software baseline. The LeNet-5 C implementation and INT8 quantization flow remain useful system-integration context, but the final board transcript used in this thesis is deliberately scoped to the reproducible CNN v1 convolution regression.

4.5.5 Performance Measurement

The NICE-accelerated convolution was benchmarked against a pure software reference implementation. For a 3×3 convolution on a 4×4 input feature map:

- CPU-only (software): 1,516 clock cycles

- NICE-accelerated (hardware): 287 clock cycles
- Speedup: $5.28\times$

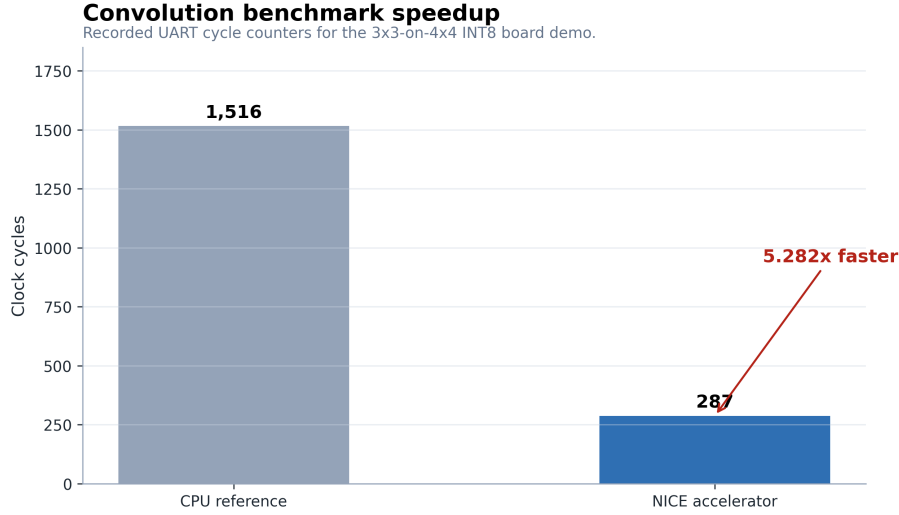


Figure 4.5: Measured convolution benchmark on the board demo: the NICE accelerator completes the test convolution in 287 cycles versus 1,516 cycles for the CPU reference, giving a $5.282\times$ speedup.

This measurement covers the convolution operation only: CLEAR, four WLOADs, four DLOADs, COMP, and RSTAT. The overall LeNet-5 inference time is dominated by the software FC layers, which perform approximately 50,000 multiply-accumulate operations per image at 16 MHz. The complete inference time per image is approximately 2–5 minutes in the current implementation, indicating that future work should target hardware acceleration of the FC layers, adoption of a Line Buffer architecture for pipelined data supply (as specified in the original task requirements), or migration to a reduced-precision FC datapath.

The measured $5.28\times$ speedup (Figure 4.5) meets the design target of $\geq 5\times$ acceleration for convolution operations.

4.6 Resource Utilization

The FPGA resource utilization for the complete E203 SoC with the CNN NICE accelerator is summarized in Table 4.6 and Figure 4.6 (post-implementation, xc7a100t).

These figures demonstrate that the complete E203 SoC with CNN accelerator fits comfortably within the A7-100T device, leaving substantial headroom for future expansion.

Table 4.6: FPGA resource utilization for the complete SoC with CNN NICE accelerator (post-implementation)

Resource	Used	Available	Utilization (%)
LUT	13,187	63,400	20.8
LUTRAM	2,843	19,200	14.8
FF	12,807	126,800	10.1
BRAM	36	135	26.7
DSP	0	240	0.0
DSP mapped to LUTs: 8-bit multipliers more efficient in fabric.			
BUFG	4	32	12.5

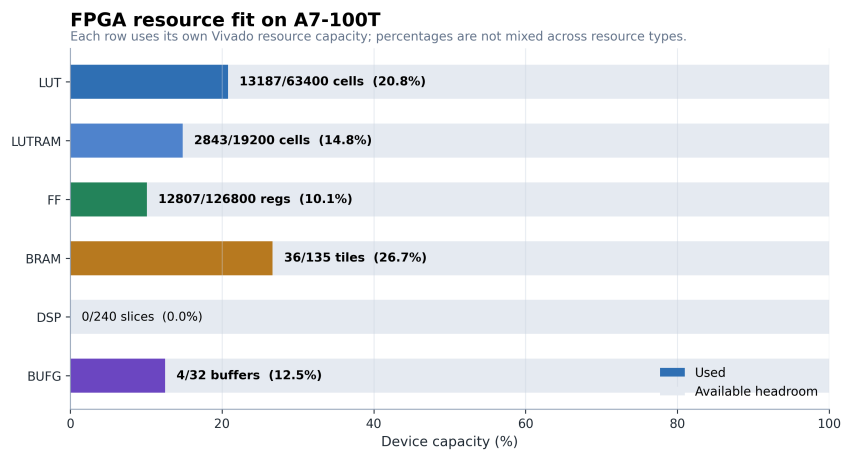


Figure 4.6: FPGA resource-capacity fit for the complete E203 SoC with CNN NICE accelerator. Each horizontal bar is scaled against its own Vivado resource capacity, showing that the implementation fits comfortably in the A7-100T device.

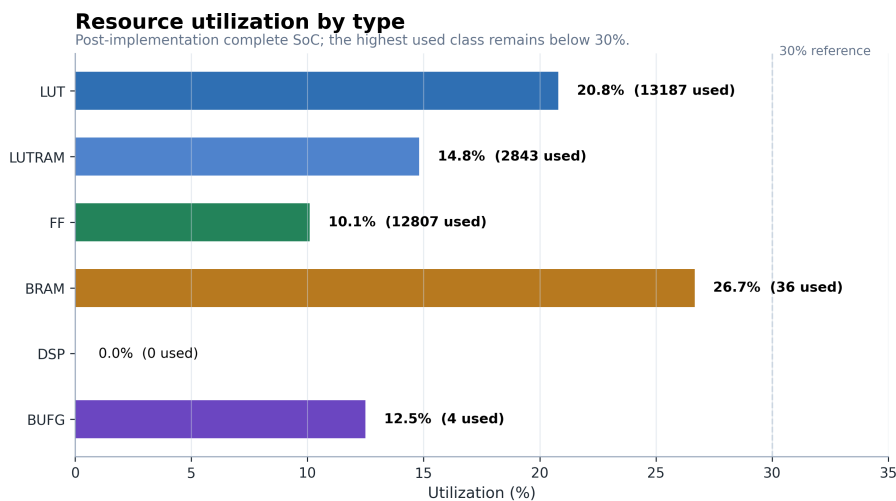


Figure 4.7: FPGA utilization by resource type for the complete SoC. LUT, FF, BRAM, DSP, and clock-buffer usage leave headroom for future accelerator expansion.

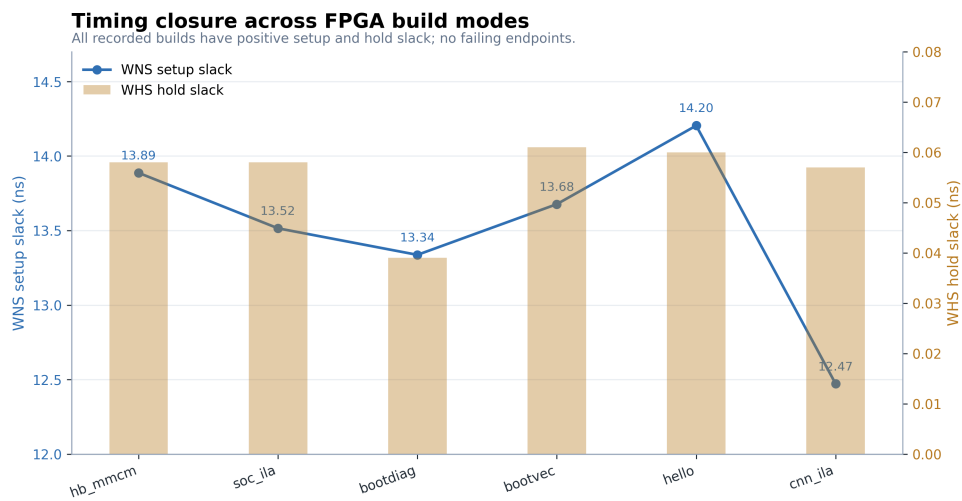


Figure 4.8: Timing closure summary across diagnostic and CNN board build configurations; all recorded builds meet setup and hold timing with positive slack.

Chapter 5

Discussion

5.1 FPGA Bring-up Challenges and Resolution

The FPGA bring-up process for the E203-based CNN accelerator system revealed several non-trivial engineering challenges that are not typically encountered in pure RTL simulation environments. These challenges and their resolutions provide valuable insights for similar SoC prototyping efforts.

5.1.1 Simulation-to-Hardware Gap

A significant finding was that the RTL simulation, once properly configured, exactly reproduced the FPGA board behavior. When the CPU failed to boot (PC=0x00000000), this behavior was observed identically in both iverilog simulation and Vivado hardware. This correlation was instrumental in enabling rapid root cause analysis, as simulation iterations take seconds compared to the 10-minute FPGA build cycle.

The key factors that caused the simulation-to-hardware gap were not related to timing, clock domain crossing, or synthesis optimization—the usual suspects in FPGA debugging. Instead, they were related to file format compatibility, macro visibility, and transport delay behavior in simulation primitives. This finding underscores the importance of establishing a verified simulation environment before committing to hardware builds.

5.1.2 ITCM/DTCM Initialization Complexity

A key discovery was that both Vivado’s synthesis-time `$readmemh` and iverilog’s simulation-time `$readmemh` interpret whitespace-separated hex tokens as full memory words. Combined

with the E203’s use of different data widths for ITCM (64-bit) and DTCM (32-bit), this created a subtle but critical initialization bug. The RISC-V `objcopy` command with `-O verilog` output produces byte-level hex, incompatible with word-level `$readmemh` for memories wider than 8 bits.

The solution—a configurable-width hex format converter—highlights the importance of understanding the specific requirements of each tool in the build chain. This issue would not manifest in an ASIC flow where memory initialization is handled through different mechanisms.

5.1.3 ILA Probe Design Strategy

The initial ILA probe configuration monitored the ITCM-dedicated bus interface, which was a natural choice because the `hello_e203` program resides in ITCM. However, the CPU’s initial boot sequence begins in mask ROM (MROM), accessed through the BIU and system memory bus rather than the ITCM interface. This mismatch caused the ILA to show no activity despite the CPU being active, leading to incorrect initial diagnosis.

The lesson for future ILA-based debugging is to include probes on all major bus interfaces during initial bring-up, rather than optimizing for the expected operational path.

5.1.4 E203 NICE Decoder Integration Constraint

A notable bring-up challenge encountered during NICE accelerator validation was the E203 decoder’s dual interpretation of the `rs2` instruction field [7]. In the standard RISC-V ISA, the `rs2` field serves a single purpose: it identifies a general-purpose register whose value is read from the register file. The E203 decoder optimizes this by skipping the register read when `rs2` encodes `x0`, since `x0` always returns zero.

However, NICE custom instructions may repurpose the `rs2` field as an immediate data operand rather than a register address. In the CNN accelerator’s `WLOAD` and `DLOAD` instructions, `rs2` encodes the PE array vector bank index (0–3). When the index is 0, the instruction’s `rs2` field is `5'b00000`, which the decoder interprets as “`rs2 = x0`, skip the register read.” The IFU retains a stale `rs2` index from the preceding instruction, and the accelerator receives incorrect operand data.

This behavior is not documented in the NICE specification, which describes the valid-ready handshake and instruction encoding but does not detail the decoder’s internal register-fetch optimization. For future NICE coprocessor designs that repurpose the `rs2` (or `rs1`) field as non-register data, the integration strategy should either (a) apply the one-line decoder patch described in Chapter 4 to unconditionally capture `rs2` for NICE instructions, or (b) ensure at the

software level that the encoded operand value never equals x0 (for example, by offsetting the index range to 1–4 instead of 0–3).

The broader lesson is that when integrating custom coprocessors into a processor core originally designed for a fixed ISA, instruction-field semantics that are optimization-safe for the base ISA may become correctness-critical when fields are repurposed. A conservative integration policy—capture all operand fields unconditionally for custom instructions—avoids such latent issues and adds negligible hardware cost (one multiplexer control bit).

5.2 NICE CNN Accelerator Validation Status

The NICE CNN accelerator test program successfully demonstrated the complete NICE instruction execution path [7] on FPGA hardware:

The CPU correctly decodes and dispatches custom-0 opcode (0x0B) instructions to the NICE coprocessor.

The NICE interface correctly handles the four-channel handshake protocol (request, response, memory, CSR).

The PE array receives weight and activation data through the WLOAD and DLOAD instructions and responds to the COMP trigger.

All NICE instructions complete without bus errors, pipeline stalls, or exception traps, as confirmed by ILA probe traces showing the CPU progressing through the full instruction sequence.

The final board-level CNN regression used a reduced 3×3 INT8 convolution over a 4×4 INT8 input. Its UART transcript shows that the NICE hardware output exactly matches both the CPU reference and the expected result (12, 23, 0, 19), and the same run reports a $5.282 \times$ kernel speedup. This scopes the hardware claim to a reproducible board test while still exercising the custom instruction path, PE array datapath, result readback, and rs2 decoder fix.

The larger LeNet-5 software and quantization flow remains a natural extension point, but the evidence package deliberately separates those software artifacts from the board transcript. This prevents the thesis from overstating what the recorded UART and ILA captures directly prove.

5.3 Comparison with Project Objectives

Table 5.1 summarizes the achievement status of each project objective.

Table 5.1: Project objective status summary

Objective	Status	Evidence
Design 4×4 PE array with NICE interface	Achieved	RTL simulation RSTAT=19
Integrate with E203 SoC	Achieved	Full-SoC simulation + FPGA build
FPGA bring-up pipeline	Achieved	6 build modes, all timing-clean
hello_e203 board validation	Achieved	UART output + ILA evidence
NICE accelerator board testing	Achieved	ILA-confirmed instruction execution + CNN v1 UART regression
Convolution speedup ($\geq 5\times$ target)	Achieved	5.28× measured (287 vs. 1516 cycles)
Full-network accuracy validation	Partial	LeNet-5 flow prepared; final board evidence scoped to reduced CNN regression

5.4 Engineering Lessons Learned

Build verification infrastructure first. The ILA probe system, simulation environment, and hex format converter were essential enablers for all subsequent debugging. Without them, the four root causes would have been extremely difficult to isolate.

Simulation is not optional for complex SoC bring-up. Even though the final target is FPGA hardware, the ability to reproduce board behavior in simulation was the single most important factor in resolving the boot failure.

Document every configuration change. The interplay between the boot-ROM macro, the prologue.tcl global defines, the fpga_mem_init.vh header, and the ITCM/DTCM hex file formats involved changes across multiple files in two repositories. Systematic documentation prevented confusion during iterative debugging.

Peripheral initialization order matters. The GPIO IOF configuration must precede UART initialization, and the UART baud rate divisor must be set before enabling the transmitter. These dependencies, while obvious in hindsight, caused several failed iterations when developing the NICE test program.

5.5 Limitations

Several limitations of the current prototype are acknowledged:

Operating Frequency. The SoC core currently operates at 16 MHz, derived from the 50 MHz board oscillator via an MMCM divider. This clock frequency was selected to match the E203 reference design’s default power-on configuration (hfextclk = 16 MHz, PLL bypass mode), simplifying the initial bring-up by avoiding PLL configuration complexity. Static tim-

ing analysis confirms substantial positive slack at 16 MHz (WNS = 24.328 ns on the clk_16M domain). The ILA sampling logic, operating directly on the 50 MHz sys_clk domain, achieves clean timing closure (WNS = 12.472 ns, zero failing endpoints across 3,730 paths), demonstrating that the Artix-7 -2 speed grade is fully capable of 50 MHz operation for the FPGA fabric. Migrating the SoC core to a higher frequency would require adjusting the MMCM output divider and verifying timing closure across the expanded core clock domain.

Inference Accuracy. The current board evidence validates a reduced CNN convolution regression rather than a full MNIST classification run. Full-network LeNet-5 accuracy should therefore be treated as a follow-up validation task unless a separate, recorded board transcript is added to the evidence package. Quantization-aware training and a documented MNIST board test would be the appropriate next steps for closing this gap.

Accelerator Coverage. Only the convolutional layers (Conv1 and Conv2) are executed on the NICE accelerator. The pooling and fully-connected layers run in software on the E203 core, which dominates the end-to-end inference latency (approximately 2–5 minutes per image at 16 MHz). Hardware acceleration of the FC layers would significantly improve throughput.

These limitations are clearly scoped and do not undermine the core contributions of the project: the complete FPGA bring-up pipeline, the systematic debugging methodology, the demonstrated execution of NICE custom instructions on FPGA hardware, and the recorded CNN v1 board regression with hardware/software result agreement and measured convolution speedup.

Chapter 6

Conclusion

6.1 Summary of Contributions

This project has successfully designed, implemented, and validated a lightweight CNN accelerator integrated with a RISC-V E203 processor [6] through the NICE custom instruction interface [7] on an FPGA platform. The principal contributions are:

CNN Accelerator Design: A 4×4 systolic PE array supporting INT8 quantized convolution was implemented with six custom NICE instructions (CFG, CLEAR, WLOAD, DLOAD, COMP, RSTAT). The design was verified through RTL simulation achieving correct RSTAT results for validated test vectors.

Complete FPGA Bring-up Pipeline: A reproducible build flow was established from RTL source through Vivado synthesis, place-and-route, and bitstream generation for the Davinci Pro A7-100T development board. Six ILA diagnostic build modes were created to support progressive hardware debugging. All builds achieved clean timing closure (WNS > 12 ns, WHS > 0.05 ns).

Hello World Board Validation: The hello_e203 bare-metal program was successfully executed on the FPGA, producing the expected three-stage UART output and confirming the complete CPU boot chain from mask ROM through ITCM execution.

Systematic Root Cause Analysis: Four critical root causes preventing CPU boot were identified and resolved through a systematic ILA-based debugging methodology: simulation DFF transport delay, ITCM/DTCM hex format incompatibility, and macro visibility in Vivado synthesis. The simulation environment was demonstrated to exactly reproduce FPGA board behavior, enabling rapid diagnosis.

NICE Accelerator Board Testing: Custom NICE test programs confirmed that the CNN

accelerator instructions execute correctly on the FPGA, with ILA evidence documenting CPU execution and memory activity. The final CNN v1 board regression shows hardware/software agreement for a 3×3 INT8 convolution and reports a measured $5.282 \times$ speedup over the CPU reference.

6.2 Key Technical Findings

The project yielded several findings of general relevance to RISC-V SoC prototyping on FPGA platforms:

ITCM/DTCM initialization format is a critical compatibility point between the RISC-V toolchain (objcopy -O verilog) and FPGA synthesis tools (\$readmemh). The byte-level hex format produced by objcopy is not directly compatible with word-level memory initialization for BRAM widths greater than 8 bits.

Macro visibility in Vivado's Verilog compilation differs from standard Verilog simulators. Macros defined in header files may not be visible to modules that do not explicitly include them, even when all files are compiled in the same project.

ILA probe placement requires understanding of the complete bus architecture. Monitoring only the expected operational path (ITCM interface) can miss activity during boot sequences that use different bus paths (BIU to system memory bus).

6.3 Future Work

Several directions for extending this work are identified:

Hardware RSTAT Optimization: Optimize the multi-tile RSTAT readback path at the hardware level to reduce per-tile overhead, further improving convolution throughput for kernels larger than 4×4 .

Accuracy Optimization: Complete a separately recorded full-network MNIST/LeNet-5 board validation flow, then explore quantization-aware training if the measured accuracy gap exceeds the project target.

FC Layer Acceleration: Extend hardware acceleration to the fully-connected layers, which currently dominate end-to-end inference latency in software (approximately 50,000 MAC operations per image at 16 MHz).

Performance Benchmarking: Conduct systematic cycle-count comparisons across representative CNN workloads to fully characterize the NICE accelerator speedup.

Design Optimization: Explore increasing the PE array size, adding pipelining within the array, and supporting additional activation functions (ReLU, leaky ReLU) through the CFG instruction.

Formal Verification: Apply formal verification methods to the NICE interface protocol to ensure correctness under all possible instruction sequences and timing conditions.

6.4 Closing Remarks

This project has demonstrated that a RISC-V-based CNN accelerator can be successfully prototyped on a commercial FPGA development board using open-source RISC-V IP and standard FPGA design tools. The systematic debugging methodology developed during this work—combining ILA-based hardware observation, RTL simulation correlation, and progressive root cause isolation—provides a reusable framework for similar SoC bring-up efforts. The evidence chain established from RTL simulation through board-level UART output represents a complete and verifiable engineering contribution.

References

- [1] V. Sze, Y.-H. Chen, T.-J. Yang, and J. S. Emer, “Efficient processing of deep neural networks: A tutorial and survey,” *Proceedings of the IEEE*, vol. 105, no. 12, pp. 2295–2329, 2017. DOI: [10.1109/JPROC.2017.2761740](https://doi.org/10.1109/JPROC.2017.2761740)
- [2] H. T. Kung, “Why systolic architectures?” *IEEE Computer*, vol. 15, no. 1, pp. 37–46, 1982. DOI: [10.1109/MC.1982.1653825](https://doi.org/10.1109/MC.1982.1653825)
- [3] A. Waterman and K. Asanović, “The RISC-V instruction set manual, volume I: Unprivileged architecture,” RISC-V Foundation, Tech. Rep. Document Version 20191213, 2019. [Online]. Available: <https://riscv.org/technical/specifications/>
- [4] A. Waterman, “Design of the RISC-V instruction set architecture,” Ph.D. dissertation, University of California, Berkeley, 2016. [Online]. Available: <https://www2.eecs.berkeley.edu/Pubs/TechRpts/2016/EECS-2016-1.html>
- [5] D. Patterson and A. Waterman, *The RISC-V Reader: An Open Architecture Atlas*. Strawberry Canyon, 2017.
- [6] Nuclei System Technology, *Nuclei hummingbird E203 processor user manual*, 2020. [Online]. Available: https://github.com/riscv-mcu/e203_hbirdv2
- [7] Nuclei System Technology, *Nuclei instruction co-extension (NICE) interface specification*, 2020. [Online]. Available: <https://doc.nucleisys.com/hbirdv2/>
- [8] A. Waterman and K. Asanović, “The RISC-V instruction set manual, volume II: Privileged architecture,” RISC-V Foundation, Tech. Rep., 2021. [Online]. Available: <https://riscv.org/technical/specifications/>
- [9] K. Asanović and D. A. Patterson, “The RISC-V instruction set architecture,” in *2014 IEEE Hot Chips 26 Symposium (HCS)*, Presentation at Hot Chips 26, 2014.
- [10] Y. LeCun, L. Bottou, Y. Bengio, and P. Haffner, “Gradient-based learning applied to document recognition,” *Proceedings of the IEEE*, vol. 86, no. 11, pp. 2278–2324, 1998. DOI: [10.1109/5.726791](https://doi.org/10.1109/5.726791)

- [11] A. Krizhevsky, I. Sutskever, and G. E. Hinton, “ImageNet classification with deep convolutional neural networks,” in *Advances in Neural Information Processing Systems 25 (NIPS)*, 2012. DOI: [10.1145/3065386](https://doi.org/10.1145/3065386)
- [12] B. Jacob et al., “Quantization and training of neural networks for efficient integer-arithmetic-only inference,” pp. 2704–2713, 2018. DOI: [10.1109/CVPR.2018.00286](https://doi.org/10.1109/CVPR.2018.00286)
- [13] R. Krishnamoorthi, *Quantizing deep convolutional neural networks: A tutorial and survey*, arXiv:1806.08342, 2018. eprint: [1806.08342](https://arxiv.org/abs/1806.08342).
- [14] S. Han, H. Mao, and W. J. Dally, “Deep compression: Compressing deep neural networks with pruning, trained quantization and Huffman coding,” in *International Conference on Learning Representations (ICLR)*, 2016.
- [15] AMD Xilinx, *Vivado design suite user guide: Synthesis (UG901)*, 2023. [Online]. Available: <https://docs.amd.com/r/en-US/ug901-vivado-synthesis>
- [16] AMD Xilinx, *Vivado design suite user guide: Implementation (UG904)*, 2023. [Online]. Available: <https://docs.amd.com/r/en-US/ug904-vivado-implementation>
- [17] AMD Xilinx, *Vivado design suite user guide: Programming and debugging (UG908)*, 2023. [Online]. Available: <https://docs.amd.com/r/en-US/ug908-vivado-programming-debugging>
- [18] 周飞燕, 金林鹏, and 董军, “卷积神经网络研究综述,” 计算机学报, vol. 40, no. 6, pp. 1229–1251, 2017. [Online]. Available: <https://kns.cnki.net/KCMS/detail/detail.aspx?dbcode=CJFQ&dbname=CJFDLAST2017&filename=JSJX201706001>
- [19] T. Chen et al., “DianNao: A small-footprint high-throughput accelerator for ubiquitous machine-learning,” in *Proceedings of the 19th International Conference on Architectural Support for Programming Languages and Operating Systems (ASPLOS)*, 2014, pp. 269–284. DOI: [10.1145/2541940.2541967](https://doi.org/10.1145/2541940.2541967)
- [20] Y. Chen, T. Chen, Z. Xu, N. Sun, and O. Temam, “Diannao family: Energy-efficient hardware accelerators for machine learning,” *Commun. ACM*, vol. 59, no. 11, pp. 105–112, 2016. DOI: [10.1145/2996864](https://doi.org/10.1145/2996864) [Online]. Available: <https://dl.acm.org/doi/10.1145/2996864>
- [21] Y.-H. Chen, T. Krishna, J. S. Emer, and V. Sze, “Eyeriss: An energy-efficient reconfigurable accelerator for deep convolutional neural networks,” *IEEE Journal of Solid-State Circuits*, vol. 52, no. 1, pp. 127–138, 2017. DOI: [10.1109/JSSC.2016.2616357](https://doi.org/10.1109/JSSC.2016.2616357)

- [22] N. P. Jouppi, C. Young, N. Patil, and D. A. Patterson, “In-datacenter performance analysis of a tensor processing unit,” in *Proceedings of the 44th Annual International Symposium on Computer Architecture (ISCA)*, 2017, pp. 1–12. DOI: [10.1145/3079856.3080246](https://doi.org/10.1145/3079856.3080246)
- [23] H. Genc et al., *Gemmini: An agile systolic array generator enabling systematic evaluations of deep-learning accelerators*, 2020. DOI: [10.1109/DAC18072.2020.9218590](https://doi.org/10.1109/DAC18072.2020.9218590)
- [24] K. Asanović et al., “The rocket chip generator,” University of California, Berkeley, Tech. Rep., 2016. [Online]. Available: <https://www2.eecs.berkeley.edu/Pubs/TechRpts/2016/EECS-2016-17.html>
- [25] 刘畅, 武延军, 吴敬征, and 赵琛, “Risc-v 指令集架构研究综述,” *软件学报*, vol. 32, no. 12, pp. 3992–4024, 2021. DOI: [10.13328/j.cnki.jos.006490](https://doi.org/10.13328/j.cnki.jos.006490) [Online]. Available: <https://link.cnki.net/doi/10.13328/j.cnki.jos.006490>
- [26] 王凯帆 et al., “香山开源高性能 risc-v 处理器设计与实现,” *计算机研究与发展*, vol. 60, no. 3, pp. 476–493, 2023. [Online]. Available: <https://kns.cnki.net/KCMS/detail/detail.aspx?dbcode=CJFQ&dbname=CJFDLAST2023&filename=JFYZ202303014>
- [27] S. Wang et al., “Optimizing cnn computation using risc-v custom instruction sets for edge platforms,” *IEEE Transactions on Computers*, vol. 73, no. 5, pp. 1371–1384, 2024. DOI: [10.1109/TC.2024.3362060](https://doi.org/10.1109/TC.2024.3362060) [Online]. Available: <https://ieeexplore.ieee.org/document/10420491/>
- [28] S. Wang, J. Zhu, Q. Wang, C. He, and T. T. Ye, “Customized instruction on risc-v for winograd-based convolution acceleration,” 2021, pp. 65–68. DOI: [10.1109/ASAP52443.2021.00018](https://doi.org/10.1109/ASAP52443.2021.00018) [Online]. Available: <https://ieeexplore.ieee.org/document/9516614/>
- [29] B. Chen, X. Wang, Y. Huang, and Z. Xu, “Riscv-fnt: A fast fnt-based risc-v processor for cnn acceleration,” 2024, pp. 292–296. DOI: [10.1109/AICAS59952.2024.10595907](https://doi.org/10.1109/AICAS59952.2024.10595907) [Online]. Available: <https://ieeexplore.ieee.org/document/10595907/>
- [30] S. B. Jigalur, C.-L. Tsai, Y.-C. Shih, and Y.-C. Kuan, “A risc-v-based high-throughput accelerator for sparse winograd cnn inference on fpga,” *IEEE Embedded Systems Letters*, vol. 17, no. 4, pp. 256–259, 2025. DOI: [10.1109/LES.2025.3531251](https://doi.org/10.1109/LES.2025.3531251) [Online]. Available: <https://ieeexplore.ieee.org/document/10845145/>

- [31] 傅思扬, 陈华, and 郁发新, “基于 risc-v 的卷积神经网络处理器设计与实现,” 微电子学与计算机, vol. 37, no. 4, pp. 49–54, 2020. DOI: [10.19304/j.cnki.issn1000-7180.2020.04.009](https://doi.org/10.19304/j.cnki.issn1000-7180.2020.04.009) [Online]. Available: <https://link.cnki.net/doi/10.19304/j.cnki.issn1000-7180.2020.04.009>
- [32] 张坤宁, 赵烁, 何虎, 邓宁, and 杨旭, “基于 risc-v 处理器的卷积加速 soc 系统设计,” 计算机工程, vol. 47, no. 4, pp. 153–157, 2021. DOI: [10.19678/j.issn.1000-3428.0057835](https://doi.org/10.19678/j.issn.1000-3428.0057835) [Online]. Available: <https://doi.org/10.19678/j.issn.1000-3428.0057835>
- [33] 廖汉松, 吴朝晖, and 李斌, “基于 risc-v 的卷积神经网络专用指令集处理器,” 计算机工程, vol. 47, no. 7, pp. 196–204, 2021. DOI: [10.19678/j.issn.1000-3428.0058371](https://doi.org/10.19678/j.issn.1000-3428.0058371) [Online]. Available: <https://doi.org/10.19678/j.issn.1000-3428.0058371>
- [34] 曹希戡, 陈鑫, and 魏同权, “基于 risc-v 的深度可分离卷积神经网络加速器,” 计算机学报, vol. 47, no. 11, pp. 2536–2551, 2024. [Online]. Available: <https://kns.cnki.net/KCMS/detail/detail.aspx?dbcode=CJFQ&dbname=CJFDLAST2024&filename=JSJX202411003>
- [35] W. Gong, F. Zhou, and F. Ge, “A multi-mode convolution coprocessor based on risc-v instruction set architecture,” 2023, pp. 1–5. DOI: [10.1109/ASICON58565.2023.10396531](https://doi.org/10.1109/ASICON58565.2023.10396531) [Online]. Available: <https://ieeexplore.ieee.org/document/10396531/>
- [36] 金华, 蔡新颖, 刘玖金, 宋雪桦, and 王昌达, “基于 fpga 的灵活量化卷积器,” 微电子学与计算机, vol. 43, no. 2, pp. 57–71, 2026. DOI: [10.19304/J.ISSN1000-7180.2024.0889](https://doi.org/10.19304/J.ISSN1000-7180.2024.0889) [Online]. Available: <https://mc.spacejournal.cn/cn/article/doi/10.19304/J.ISSN1000-7180.2024.0889>
- [37] H. Wu, J. Li, and X. Chen, “Implementation of cnn heterogeneous scheme based on domestic fpga with risc-v soft core cpu,” 2022, pp. 158–159. DOI: [10.1109/ICTA56932.2022.9963056](https://doi.org/10.1109/ICTA56932.2022.9963056) [Online]. Available: <https://ieeexplore.ieee.org/document/9963056/>
- [38] S. Feng, J. Wu, S. Zhou, and R. Li, “The implementation of lenet-5 with nvdla on risc-v soc,” 2019, pp. 39–42. DOI: [10.1109/ICSESS47205.2019.9040769](https://doi.org/10.1109/ICSESS47205.2019.9040769) [Online]. Available: <https://ieeexplore.ieee.org/document/9040769/>
- [39] I. Kuon and J. Rose, “Measuring the gap between FPGAs and ASICs,” *IEEE Transactions on Computer-Aided Design of Integrated Circuits and Systems*, vol. 26, no. 2, pp. 203–215, 2007. DOI: [10.1109/TCAD.2007.891375](https://doi.org/10.1109/TCAD.2007.891375)

- [40] Nuclei System Technology, *Nuclei SDK documentation*, 2020. [Online]. Available: <https://github.com/riscv-mcu/nuclei-sdk>
- [41] Nuclei System Technology, “Nuclei E203 debug subsystem technical report,” Tech. Rep., 2021. [Online]. Available: <https://doc.nucleisys.com/hbirdv2/>
- [42] GDB Contributors, *Debugging with GDB*, 2023. [Online]. Available: <https://sourceware.org/gdb/documentation/>
- [43] OpenOCD Contributors, *OpenOCD user’s guide*, 2023. [Online]. Available: <https://openocd.org/doc/html/index.html>
- [44] ALIENTEK, *Davinci Pro FPGA development board user manual*, 2022. [Online]. Available: <http://www.openedv.com/docs/boards/fpga/>